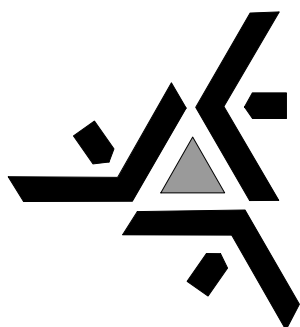




Universidade Estadual de Maringá
Centro de Tecnologia
Departamento de Informática

Estudo de Princípios para Modelagem Orientada a Aspectos

Igor Fábio Steinmacher

TG-17-02

Maringá - Paraná

Brasil

Universidade Estadual de Maringá
Centro de Tecnologia
Departamento de Informática

Estudo de Princípios para Modelagem Orientada a Aspectos

Igor Fábio Steinmacher

TG-17-02

Trabalho de Graduação apresentado ao Curso de
Ciência da Computação, do Centro de Tecnologia, da
Universidade Estadual de Maringá.
Orientador: *Profa. Dra. Itana Maria de Souza Gimenes*

**Maringá - Paraná
2003**

Igor Fábio Steinmacher

Estudo de Princípios para Modelagem Orientada a Aspectos

Este exemplar corresponde à redação final da monografia aprovada como requisito parcial para obtenção do grau de Bacharel em Ciência da Computação da Universidade Estadual de Maringá, pela comissão formada pelos professores:

Orientador: Prof^ª. Itana Maria de Souza Gimenes
Departamento de Informática, CTC, DIN

Prof^ª. Elisa Hatsue Moriya Huzita
Departamento de Informática, CTC, DIN

Prof^ª. Tânia Fátima Calvi Tait
Departamento de Informática, CTC, DIN

Maringá, Maio de 2003

Agradecimentos

Gostaria agradecer primeiramente a Deus, por ter me concedido a oportunidade de concluir este trabalho, com o qual finalizo mais uma fase de minha vida. Gostaria de dedicar esta conquista aos meus pais que não mediram esforços para me manter durante o tempo que estive cursando a faculdade, Pai e Mãe vocês foram fantásticos, amo vocês! Um grande abraço ao meu irmão, à minha irmã e aos meus sobrinhos, que são parte de minha vida. Denerval, obrigado por todos os conselhos e ajuda que necessitei durante o curso.

Agradeço ainda à minha orientadora, Itana, que me orienta há 3 anos, e que me ajudou em grande parte a conseguir o que tenho a nível acadêmico hoje. Agradeço ainda ao ex-aluno do DIN, hoje doutorando na PUC-Rio, Alessandro Garcia, que muito me auxiliou no presente trabalho. Citando os que me ajudaram neste trabalho, posso nomear ainda Parnas, Dijkstra, Tanenbaum, Kickzales e Von Neumann.

Gostaria de estender meus agradecimentos a todos os que compartilharam este tempo comigo dentro de sala de aula ou nos laboratórios. Sou grato também ao corpo docente do Departamento de Informática, agradeço a todos aqueles que foram meus grandes mestres, que me ensinaram muito, a todos aqueles que foram grandes amigos, com quem podia contar em grandes momentos, e agradeço também aos que foram apenas meus professores.

Agradecimentos especiais aos que não foram apenas colegas dentro da universidade, aos meus amigos: Cefernan, Marim, Jorge, Kengo, Flávio, Marcel, Zeddy, Trooper, Copas, Marco Aurélio “_”, Sílvia e Kiti (*in memorian*), que não se contentavam em estarem unidos apenas dentro da sala ou do departamento, mas momentos extra UEM, churrascos, mesas de bar, festas e apoio em momentos em que se fez necessário. Gostaria de expressar meus agradecimentos também à AMBEV, que me ajudou em alguns momentos.

Para concluir abraços ao pessoal do Rotaract, do Coral, da OAE e do Grupo Escoteiro, que me trouxeram bons momentos e onde eu me apoiava nos momentos de fraqueza.

Resumo

Linguagens procedimentais e orientadas a objetos não são suficientemente claras para implementar algumas decisões de projeto. Essas linguagens não representam de maneira eficiente estas decisões, consequentemente, dificultam o desenvolvimento e manutenção do software. A necessidade de técnicas que possam apoiar o programador na atividade de separar objetos e aspectos, originou a Programação Orientada a Aspectos (*AOP – Aspect Oriented Programming*). Aspectos são justamente as propriedades do sistema que se espalham por seus vários componentes (*crosscut*). Exemplos dessas propriedades são: logging, tratamentos de erros, políticas de sincronização e rastreamento (*tracing*). Com a introdução de aspectos, faz-se necessária uma posterior composição com os componentes sobre os quais eles influenciarão. Um conceito importante no processo de composição são os pontos de combinação, propriedades comuns aos componentes do sistema através dos quais os aspectos se coordenam. A identificação e estruturação de componentes, aspectos, pontos de combinação e relacionamentos, assim como a captura de conflitos potenciais, requerem um método de modelagem apropriado, além das ferramentas de implementação. Um ponto de grande complexidade na orientação a aspectos é a existência de dois níveis de decomposição (componentes e aspectos). Para abordar este problema, necessita-se de uma modelagem que possa fornecer um conjunto de diretivas, auxiliando o desenvolvedor na identificação e composição de aspectos e componentes. Esta modelagem seria como uma extensão do modelo orientado a objetos, adicionando-se à modelagem de aspectos e *crosscuttings*. Com isso, se espera um modelo que facilite a especificação da estrutura de sistemas orientados a objetos, com aspectos e *crosscuttings* promovendo um fácil entendimento, consistência e redução da complexidade na concretização da modelagem através da implementação em linguagens de aspectos. Este trabalho tem como objetivos estudar a programação orientada a aspectos, investigando quais os conceitos relevantes para a modelagem de sistemas. Pretende-se identificar aspectos em uma aplicação orientada a objetos existente e implementar os aspectos identificados em uma linguagem de programação orientada a aspectos. Finalmente, pretende-se identificar princípios básicos para um método de modelagem orientado a aspectos.

Abstract

Procedural and object oriented programming techniques are not sufficiently clear to implement some project decisions. These languages do not clearly represent these decisions, consequently, turning the development and the software maintenance harder. The demand for techniques that can support the programmer in the activity of separating objects and aspects, has originated Aspect Oriented Programming (AOP). Aspects are exactly the system properties that are scattered throughout the code (crosscut). Some examples of these properties are: logging, synchronization policies and tracing. Introducing aspects, a composition between the aspect and the components that it crosscuts become necessary. An important concept in the composition process is the join points, the elements of the component language semantics that the aspect programs coordinate with. Identifying and structuring components, aspects, join points and relationships, and capturing conflicts, require an appropriated design model, besides the implementation tools. A high complexity point on Aspect Orientation is the existence of two decomposition levels (components and aspects). To solve this problem is necessary a modeling method that provides a set of directives, assisting developers on identification and composition of aspects and components. This method would be an Object orientation paradigm extension, adding aspects and crosscutting modeling. On this way, a new model that facilitates OO systems specification with aspects and crosscutting is expected, promoting an easy agreement, consistence and complexity reduction on modeling systems and implementing them on aspect based languages. On this context, these paperwork objectives are to study Aspect Oriented Programming, investigating important concepts during the system design modeling. Is pretended to identify aspects on an Object Oriented application and implement them on an aspect language. Finally, is pretended to identify useful principles on an aspect oriented modeling method.

Índice

CAPÍTULO 1	1
INTRODUÇÃO	1
1.1 ORGANIZAÇÃO DO TRABALHO	2
CAPÍTULO 2	3
ABORDAGEM ORIENTADA A ASPECTOS.....	3
2.1 CONCEITOS BÁSICOS	4
2.1.1 Aspectos.....	4
2.1.2 Fundamentos da Programação Orientada a Aspectos.....	5
2.1.3 Combinador de Aspectos	6
2.1.4 Propósito de uma Linguagem de Aspectos.....	8
2.2 TECNOLOGIA DE SUPORTE À PROGRAMAÇÃO ORIENTADA A ASPECTO	9
2.2.1 HyperJ.....	9
2.2.2 AOP/ST.....	11
2.2.3 D.....	11
2.2.4 AspectJ.....	14
2.2.5 AspectC.....	15
2.2.6 AspectC++	16
2.2.7 AspectS	17
2.2.8 Comparativo.....	18
2.3 CONSIDERAÇÕES FINAIS	19
CAPÍTULO 3	20
PROPOSTA DE MODELAGEM ORIENTADA A ASPECTOS	20
3.1 <i>UML</i>	20
3.2 <i>UML PROFILE</i> PARA MODELAR ASPECTOS EM <i>ASPECTJ</i>	21
3.2.1 Abstrações Básicas do AspectJ.....	21
3.2.2 Modelando Aspectos em AspectJ	22
3.2.3 Trabalhos Relacionados	25
3.3 CONSIDERAÇÕES FINAIS	29
CAPÍTULO 4	31
ESTUDO DE CASO	31
4.1 MODELANDO SISTEMAS ORIENTADOS A ASPECTOS.....	31
4.1.1 Estudo de Caso: Hello World	31
4.1.2 Estudo de Caso: Tracing	35
4.1.3 Uma análise comparativa.....	41
4.2 CONSIDERAÇÕES FINAIS	42
CAPÍTULO 5	44
CONCLUSÕES.....	44

REFERÊNCIAS BIBLIOGRÁFICAS.....46

BIBLIOGRAFIA50

APÊNDICE A.....52

CÓDIGO FONTE TRACING – <i>JAVA</i>.....	52
A.1 CLASSE FIGURA. <i>JAVA</i>	52
A.2 CLASSE QUADRADO. <i>JAVA</i>	53
A.3 CLASSE CIRCULO. <i>JAVA</i>	54
A.4 CLASSE TRACE. <i>JAVA</i>	55
A.5 CLASSE EXEMPLO. <i>JAVA</i>	56

APÊNDICE B.....57

CÓDIGO FONTE TRACING – <i>ASPECTJ</i>.....	57
B.1 CLASSE FIGURA. <i>JAVA</i>	57
B.2 CLASSE QUADRADO. <i>JAVA</i>	58
B.3 CLASSE CIRCULO. <i>JAVA</i>	59
B.4 ASPECTO TRACE. <i>JAVA</i>	60
B.5 ASPECTO TRACECLASSES. <i>JAVA</i>	61
B.6 CLASSE EXEMPLO. <i>JAVA</i>	62

APÊNDICE C.....63

CÓDIGO FONTE HELLO WORLD – <i>ASPECTJ</i>	63
C.1 CLASSE HELLOWORLD. <i>JAVA</i>	63
C.2 ASPECTO SAUDACAO. <i>JAVA</i>	63
C.3 ASPECTO SAUDJAPONESA. <i>JAVA</i>	63

Índice de Figuras

Figura 2.1 – Um sistema implementado com e sem a utilização de aspectos	4
Figura 2.2 – Estrutura de um Programa Orientado a Aspectos	6
Figura 2.3 – Protocolo de um coordenador COOL [LOP97]	12
Figura 2.4 – Protocolo de um portal RIDL [LOP97]	13
Figura 2.5 – Arquitetura do compilador <i>AspectC++</i> [GAL01]	17
Figura 3.1 – Representando uma Classe com campo para o Aspecto	23
Figura 3.2 – Representando um Aspecto	24
Figura 3.3 – Representando um <i>pointcut</i> e seus <i>advices</i>	24
Figura 3.4 – Relacionando Classes, Aspectos e seus Pointcuts	25
Figura 4.1 – Classe HelloWorld	32
Figura 4.2 – Aspecto Saudação	32
Figura 4.3 – Modelagem para o programa <i>HelloWorld</i> Orientado a Aspectos	33
Figura 4.4 – Adicionando aspecto Saudação Japonesa	34
Figura 4.5 – Sistema com a inserção de outro aspecto	34
Figura 4.6 – Modelagem de um sistema Orientado a Objetos com <i>tracing</i>	36
Figura 4.7 – Classe Figura com código de <i>Tracing</i> entrelaçado	37
Figura 4.8 – Modelagem do sistema Orientado a Aspectos com Tracing	39
Figura 4.9 – Implementação de um aspecto modelado	40
Figura 4.10 – Aspecto <i>TraceClasse</i> como <i>extend</i> do aspecto <i>Trace</i>	41

Índice de Tabelas

Tabela 2.1 – Comparação de Linguagens e Ferramentas com suporte a aspectos	18
Tabela 4.1 – Comparativo AOP x OOP	41

Capítulo 1

Introdução

A preocupação em se produzir software de qualidade tem aumentado o uso da orientação a objetos, que oferece maiores níveis de reutilização e manutenibilidade. Entretanto, este paradigma tem demonstrado algumas limitações [OSS99], como o entrelaçamento (*code tangling*) e o espalhamento de código, fazendo com que certos trechos de código do sistema se repitam em vários módulos.

Estes fenômenos tornam a implementação utilizando linguagens orientadas a objetos complexa, dificultando a separação das funcionalidades básicas do sistema de funções específicas. Isto torna os sistemas mais complexos, uma vez que a dependência entre os componentes funcionais aumenta, fugindo de sua funcionalidade principal e tornando-os menos reutilizáveis e de mais difícil manutenção.

Neste sentido, começaram a surgir as extensões do paradigma orientado a objetos, o que é chamado de *Post-Objects Programming (POP)* [ELR01a]. Dentro destas encontramos a *Aspect Oriented Programming (AOP)*, que é o foco do presente trabalho. Estas técnicas tentam solucionar ou reduzir as limitações já descritas.

A Programação Orientada a Aspectos (AOP) aumenta a modularidade, separando o código que implementa funções diretamente ligadas ao sistema das ditas preocupações ortogonais (*crosscutting concerns*) ao sistema. Essas preocupações afetam diferentes partes do sistema, tais como persistência, *logging* e tratamento de exceções. O aumento da modularidade implica em sistemas mais legíveis, os quais são facilmente projetados e mantidos.

O presente trabalho tem como objetivo o estudo da Programação Orientada a Aspectos, investigando quais os conceitos relevantes para a modelagem de sistemas, além de identificar princípios básicos para métodos de modelagem orientado a aspectos. Com isto,

define-se uma proposta de modelagem que suporte a orientação a aspectos como uma extensão da orientação a objetos.

A proposta de modelagem definida tem como intenção prover meios para se converter um sistema, originalmente orientado a objetos, em um sistema baseado no paradigma da orientação a aspectos. Afim de testar a modelagem, são propostos alguns sistemas para serem usados como estudo de caso, que serão modelados e implementados. Para investigar os aspectos de implementação utilizaremos a ferramenta *AspectJ* [ASP03], que foi desenvolvida pela *Xerox Company* como sendo uma extensão da linguagem *JAVA*.

1.1 Organização do trabalho

O trabalho está dividido em 4 capítulos, conforme segue.

O capítulo 2 contém a introdução aos conceitos básicos relacionados à Orientação a Aspectos, além da apresentação de algumas linguagens e ferramentas de suporte à AOP.

No capítulo 3 são estudados princípios para modelagem orientada a aspectos, com a proposta de uma extensão à *UML* com suporte à AOP.

O capítulo 4 traz os estudos de caso que utiliza a proposta de modelagem introduzida no capítulo 3 para modelar um sistema inicialmente orientado a objetos. Após a modelagem este sistema é implementado utilizando-se *AspectJ*, uma extensão da linguagem *Java* com suporte a aspectos.

No capítulo 5 são apresentadas as conclusões deste trabalho.

Capítulo 2

Abordagem Orientada a Aspectos

O surgimento do paradigma da Programação Pós-Objetos (POP) traz técnicas que buscam incrementar o poder do paradigma da Orientação a Objetos, e tem se tornado um campo fértil para pesquisas [ELR01a]. Dentre as técnicas podemos citar a *Aspect-Oriented Programming (AOP)*, a *Subject-Oriented Programming* [OSS99] e a *Adaptive Programming*. Estas técnicas visam o desenvolvimento de sistemas em módulos cada vez menores, criando módulos em situações em que a Orientação a Objetos tem se demonstrado limitada.

A Programação Orientada a Aspectos (AOP) parece bastante promissora [MUR01]. Esta técnica se baseia na idéia de que os sistemas são melhor programados através da separação de preocupações (*separation of concerns*), com as respectivas descrições dos seus relacionamentos. Isto resulta na concepção de um sistema que separa o conjunto de componentes, expressos em uma linguagem de programação atual, do conjunto de preocupações ortogonais (*crosscutting concerns*) identificadas.

Pode-se enxergar um sistema complexo como uma implementação de várias preocupações (*concerns*). Um sistema comum consiste de vários tipos de preocupações, incluindo o código de negócios, logging, tratamento de exceções e persistência. A Figura 2.1 ilustra estas preocupações com e sem a utilização da AOP.

Assim, apresentamos neste capítulo os conceitos básicos sobre a Abordagem Orientada a Aspectos, bem como algumas ferramentas e linguagens de suporte à AOP. Estes conceitos são aplicados a um estudo de caso, visando experimentar a aplicação destes.

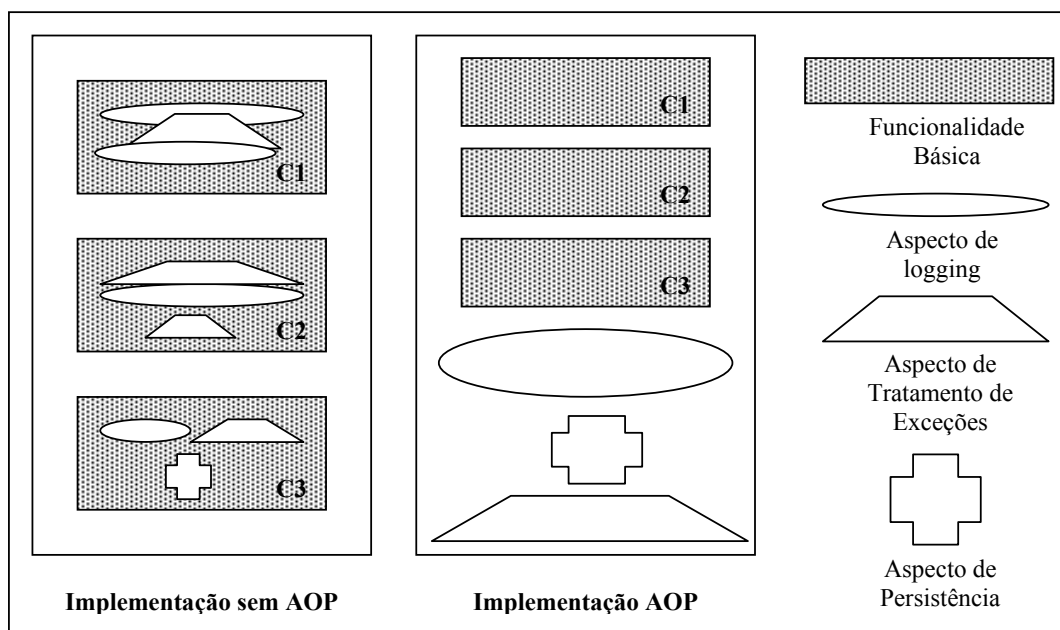


Figura 2.1 – Um sistema implementado com e sem a utilização de aspectos

2.1 Conceitos Básicos

Kickzales [KIC97] *et al* propõem agrupar as linguagens orientadas a objetos, procedimentais e funcionais como linguagens de procedimento generalizado, já que estas possuem uma raiz comum em seus mecanismos de abstração e composição. Os métodos de modelagem que têm surgido para estas linguagens tendem a dividir o sistema em unidades de comportamento ou em funções. A este estilo se dá o nome de decomposição funcional [PAR72]. Em cada um dos paradigmas as unidades são encapsuladas de maneira diferente (procedimento, funções, objetos). O problema aparece quando percebe-se que o que é encapsulado são as unidades funcionais do sistema, desconsiderando algumas outras questões importantes não localizadas no projeto funcional.

2.1.1 Aspectos

Ao utilizar-se somente das linguagens de procedimento generalizado, o programador terá de realizar a tarefa de compor as propriedades que se entrelaçam, o que leva a um aumento na complexidade de implementação e na legibilidade do código. Neste contexto se inserem os aspectos. Segundo Kickzales [KIC97] *et al* uma propriedade de um sistema que deve ser implementada pode ser vista como um componente adicional ou um aspecto:

- **Um componente** pode ser claramente encapsulado em um procedimento generalizado. Os componentes tendem a ser unidades de decomposição funcional do sistema, como contas bancárias e filtros de imagens.
- **Um aspecto** não pode ser claramente encapsulado em um procedimento generalizado. Aspectos não são normalmente unidades da decomposição funcional de um sistema. Estes afetam a semântica dos componentes sistematicamente. Como exemplo tem-se controle de concorrência em operações em uma mesma conta bancária, logging e sincronização de objetos concorrentes.

Tendo estas definições é possível definir um aspecto como sendo uma propriedade de um sistema que não pode ser encapsulada claramente e envolve diversos componentes funcionais.

2.1.2 Fundamentos da Programação Orientada a Aspectos

Os principais requisitos para uma implementação baseada na Programação Orientada a Aspectos são:

- uma linguagem para definir as funcionalidades básicas do sistema, conhecida como linguagem de componentes. Por exemplo *C++*, *Java*, *LISP*, *Cobol*;
- uma ou várias linguagens de aspectos, para especificar os aspectos. Como exemplo: *COOL*, *RIDL*, *HyperJ*, *AspectJ*;
- um combinador de aspectos (*weaver*), para combinar as linguagens;
- um programa escrito em linguagem de componentes;
- um ou mais programas de aspectos que implementam aspectos.

Graficamente pode-se observar uma estrutura conforme mostra a Figura 2.2.

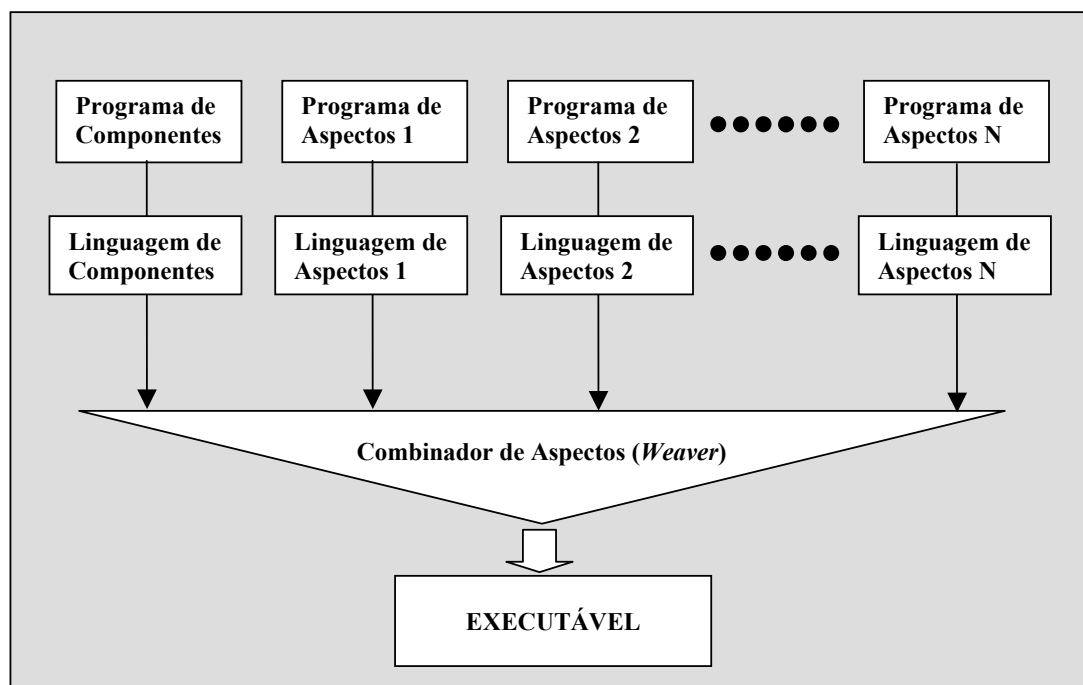


Figura 2.2 – Estrutura de um Programa Orientado a Aspectos

Para se modelar um sistema baseado em aspectos é necessário distinguir o que se deve implementar em linguagem de componentes, o que se deve implementar em linguagem de aspectos e o que deve ser compartilhado entre as duas linguagens. As linguagens de aspectos devem fornecer meios para a implementação dos aspectos de maneira intuitiva, natural e concisa [KIC97].

Para se desenvolver um sistema orientado a aspectos é necessário primeiramente decompor os requisitos para distinguir os componentes dos aspectos. Em seguida deve-se implementar os requisitos em separado. Finalmente, as regras de composição devem ser escritas, para combinar o sistema por completo.

2.1.3 *Combinador de Aspectos*

A combinação realizada pelo combinador de aspectos resulta do cruzamento de aspectos com os componentes da linguagem de componentes. A função deste último é processar a linguagem de aspectos juntamente com a de componentes, compondo as duas de forma correta produzindo a interação desejada. Esta interação não é a mesma que ocorre entre os

módulos da linguagem de componentes. Para isto, é necessário introduzir o conceito de pontos de combinação (*join points*).

Os pontos de combinação oferecem a interface entre os aspectos e os componentes. São locais dentro do código onde é possível agregar o comportamento que destaca a AOP. Este comportamento é especificado como sendo um aspecto. Pontos de combinação podem ser chamadas a métodos, construtores, destrutores ou chamadas a funções.

O projeto de um sistema orientado a aspectos requer o entendimento sobre o que deve ser descrito na linguagem de componentes e de aspectos, bem como as características que são compartilhadas entre elas [PIV01]. Mesmo sendo as duas linguagens diferentes, é necessário que elas possuam meios que possibilitem ao combinador de aspectos mesclar os dois tipos de linguagens.

Para realizar sua função, o combinador percorre o código de aspectos, recolhe todas as informações dos pontos de combinação. Com estas informações localiza, então, os pontos de combinação, combinando o código de maneira a implementar o que é especificado nas duas linguagens.

Quando se quer implementar um sistema baseado no paradigma da Orientação a Aspectos é necessário decidir a forma de entrelaçar a linguagem de componentes com a linguagem de aspectos. O entrelaçamento pode ser feito através da combinação estática ou da combinação dinâmica.

Um processo de combinação estático consiste em modificar o código fonte escrito em linguagem de componentes, inserindo sentenças nos pontos de combinação, que representam o código de aspectos. Um exemplo deste tipo de combinador é o combinador do *AspectJ*.

Num processo estático a combinação entre aspectos e componentes é realizada em tempo de compilação. O resultado obtido com este tipo de combinação é comparável com programas escritos sem uso de técnica orientada a aspectos, em termos de desempenho. Este combinador evita que o nível de abstração introduzido pela AOP cause um impacto

negativo no desempenho da aplicação, já que tudo se realiza em tempo de compilação, inexistindo a sobrecarga na execução. Outra vantagem é com relação à segurança, pois são evitados erros em tempo de execução. Uma desvantagem da combinação estática se deve ao fato de que os aspectos são ‘fixos’, não podendo ser modificados em tempo de execução.

No processo dinâmico todas as combinações são feitas em tempo de execução. Neste tipo de combinação os aspectos devem existir em tempo de compilação e em tempo de execução. Para conseguir isto, os aspectos e os componentes entrelaçados devem ser modeladas como objetos e manterem-se no executável. Um combinador deste tipo pode adicionar, remover e adaptar aspectos em tempo de execução. Como exemplo deste tipo de combinador pode-se citar o *AOP/ST* e *AspectS*.

Utilizando-se um combinador dinâmico o processo de combinação se realiza em tempo de execução, prejudicando o desempenho da aplicação. Este tipo de combinação oferece maior flexibilidade ao programador, pois existe a possibilidade de se modificar, alterar ou remover um aspecto segundo informações geradas em tempo de execução. A possibilidade de alteração em tempo de execução diminui a segurança do sistema, pois, por exemplo, todas as propriedades de um determinado aspecto são removidas e em seguida uma destas propriedades é solicitada pela aplicação.

2.1.4 Propósito de uma Linguagem de Aspectos

Uma linguagem de aspectos pode ter dois enfoques diferentes: linguagens de propósito específico e linguagens de propósito geral.

As linguagens de aspectos de propósito específico recebem este nome visto que são capazes de tratar exclusivamente um ou alguns aspectos específicos, como sincronismo e tratamento de exceções, não possuindo suporte a qualquer outro tipo de aspectos. Têm um nível de abstração mais alto que a linguagem de componentes para a qual foi implementada. Para garantir a utilização dos aspectos para os quais a linguagem foram modeladas, estas linguagens, geralmente, impõem restrições na utilização da linguagem de componentes. A linguagem de aspectos pode coibir o uso de alguns métodos e palavras

reservadas da linguagem de componentes para evitar que esta última seja capaz de implementar as preocupações, que seriam os aspectos.

As linguagens de aspectos de propósito geral são modeladas com a finalidade de poder implementar qualquer tipo de aspecto. Não é possível implementar restrições junto à linguagem de componentes, pois, geralmente, ambas possuem o mesmo conjunto de instruções. Como exemplo pode-se citar a linguagem *AspectJ*, que tem o *Java* como linguagem de componentes.

As linguagens de propósito geral provêm um ambiente mais familiar para desenvolvimento, uma vez que utiliza, geralmente, os mesmos ambientes utilizados por suas linguagens de componentes. É capaz de implementar todos os aspectos que se possa encontrar em um sistema, sem ter de contar com problemas de trocar e ferramentas para cada aspecto a ser implementado. Não é necessário o aprendizado de uma nova linguagem a cada aspecto que se implemente. Se o que se deseja é implementar um aspecto para uma única aplicação, então uma linguagem de aspectos de propósito específico terá um melhor desempenho, com uma separação de preocupações mais clara. Porém, quando se quer implementar várias preocupações (*concerns*) em uma aplicação encontramos o problema da inserção de novas restrições à linguagem de componentes a cada linguagem de aspectos que se necessite, podendo desencadear erros naqueles já implementados.

2.2 Tecnologia de suporte à Programação Orientada a Aspecto

Nesta seção são descritas sucintamente algumas linguagens e ferramentas orientadas a aspectos disponíveis atualmente.

2.2.1 *HyperJ*

No *HyperJ* [HYP03] as preocupações (*concerns*) são agrupados em uma estrutura multidimensional. Para analisar a fundo esta ferramenta é necessário o conceito de *MDSoc* (*Multi Dimensional Separation of Concerns*), introduzido por [OSS99]:

- um *concern space* concentra todas as unidades em um conjunto de sistemas ou bibliotecas. Este espaço tem como função organizar as unidades neste conjunto de sistemas para separar todas as preocupações importantes, descrevendo as inter-relações entre as preocupações e indica como os componentes do software e o resto do sistema podem ser construídos a partir das unidades que especificam estas preocupações. Pode-se, assim, identificar três componentes para separar preocupações (*separation of concerns*): identificação, encapsulamento e integração.
- HiperEspaço (*hyperSpace*) é um *concern space* estruturado para apoiar separação multidimensional de preocupações. Sua principal característica é que as unidades se organizam em uma matriz onde cada eixo representa uma dimensão de preocupação, e cada ponto uma preocupação naquela dimensão.
- *HyperSlices* são blocos construtores que podem integrar-se para formar um outro bloco construtor maior, e até um sistema.
- HiperMódulo (*HyperModule*) é um conjunto de *hyperslices* e um conjunto de regras de integração que especificam como os *hyperslices* se relacionam entre si e como devem se integrar.

Esta ferramenta suporta separação multidimensional de preocupações (*MDSoc*) em *Java*. *HyperJ* permite compor um conjunto de modelos separado, em que cada um encapsula uma preocupação definindo e implementando uma hierarquia de classes apropriada para esta preocupação. Cada modelo deve se estender por si próprio, aumentando seu comportamento juntando-se com outro.

HyperJ inclui uma ferramenta que permite identificar preocupações, especificar módulos e sintetizar sistemas e componentes. A ferramenta oferece um módulo para a edição e construção de regras de integração, permitindo um processo simples de teste para a integração de preocupações no sistema. O usuário, primeiramente, cria hiper módulos escolhendo um conjunto de preocupações e um conjunto de regras de integração. *HyperJ* cria *hyperslices* válidos de acordo com as escolhas. O usuário pode inserir novas regras ou modificar as existentes e assim continuar o processo de refinamento até obter o modelo desejado.

Os principais benefícios que *HyperJ* traz são:

- separação e modularização de preocupações de maneira flexível;
- composição não invasiva com adaptação de componentes;
- possibilita a criação de extensões e configuração de componentes sem modificação do código fonte;
- desenvolvimento descentralizado de classes.

2.2.2 AOP/ST

AOP/ST é uma ferramenta que provê suporte AOP com combinação de aspectos dinâmicos para *Visualworks / SmallTalk*. Ela é composta por um combinador para *SmallTalk* e duas linguagens de aspectos: uma para sincronização de processos e uma para acompanhamento de fluxo de execução de um programa (*tracing*).

O combinador de *AOP/ST* é aberto para a integração com outras linguagens de aspectos, bem como possui um conjunto de componentes que possibilitam o desenvolvedor criar suas próprias linguagens de aspectos.

2.2.3 D

D [LOP97] é um ambiente de linguagens de aspectos que fornece abstrações para a implementação de esquemas de sincronização e transferência remota de dados em sistemas distribuídos. Se chama ambiente pois na verdade oferece duas linguagens: *COOL*, para controlar a sincronização de *threads*, e *RIDL*, para programar a transferência entre componentes remotos. Essas linguagens foram desenvolvidas de maneira independente de uma linguagem de componentes.

Como *D* impõe algumas restrições sobre a linguagem de componentes, podemos dizer que ela é semi-independente de linguagem de componentes. Essas restrições são satisfeitas pelas linguagens Orientadas a Objetos mais conhecidas (*C++*, *SmallTalk*, *Java* ou *Eiffel*).

Os aspectos descritos em *D* afetam o comportamento distribuído e concorrente dos componentes, deixando o programador livre para se preocupar com a funcionalidade dos

componentes, depois disso é que deve descrever como os aspectos afetam os componentes, separadamente ao restante do sistema.

COOL (Coordination Aspect Language)

É uma linguagem de aspectos de domínio específico para tratar a exclusão mútua de *threads*, sincronização, suspensão e reativação de *threads*. Isto é feito de maneira separada do código referente às classes do sistema. Um programa *COOL* é uma coleção de módulos coordenadores. Estes módulos são associados a classes, podendo estar associado a uma ou mais classes. Os coordenadores têm por responsabilidade a sincronização das *threads* através da execução de métodos das classes. Eles são associados às instâncias das classes assim que estas são criados, e durante o tempo de vida destes objetos esta relação tem um protocolo bem definido.

Um coordenador associado a uma classe pode visualizar todas as variáveis e métodos públicos, protegidos e privados desta classe. Pode também visualizar todas as variáveis e métodos não privados da suas superclasses.

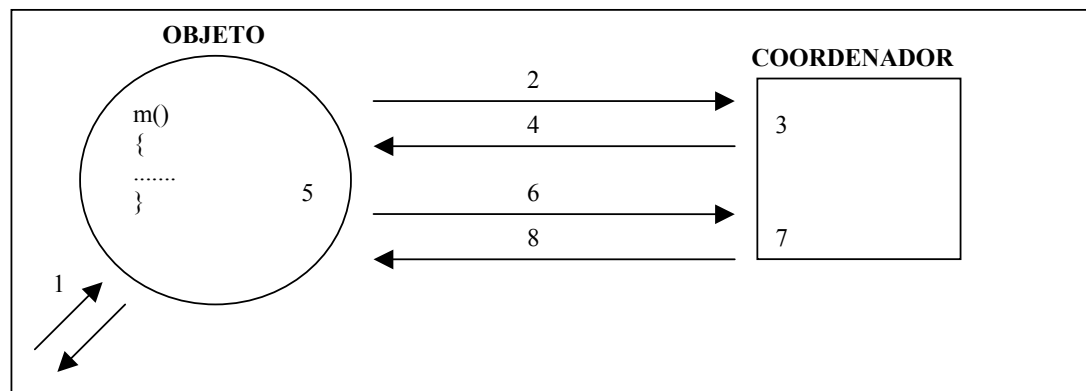


Figura 2.3 – Protocolo de um coordenador *COOL* [LOP97]

O protocolo da Figura 2.3 é definido por [LOP97] através dos seguintes passos:

1. Dentro de uma *thread* T, ocorre uma invocação ao método m do objeto.
2. A mensagem é enviada ao coordenador do objeto.
3. O coordenador verifica restrições de exclusão e pré-condições para o método m. Se alguma das restrições não é satisfeita T é suspensa. Quando todas as restrições são

atendidas o coordenador executa as sentenças definidas no bloco *on_entry* para o método *m*.

4. A requisição é enviada para o objeto.
5. O método *m* é executado pela *thread T*.
6. Quando a invocação retornar o resultado é apresentado ao coordenador.
7. O coordenador executa as sentenças definidas no bloco *on_exit* para o método *m*.
8. A invocação do método retorna.

RIDL (Remote Interaction and Data transfers aspect Language)

É uma linguagem de aspectos de domínio específico que fornece mecanismos para a manipulação de dados entre diferentes espaços de execução separadamente dos componentes funcionais da aplicação.

Um programa *RIDL* conta com um conjunto de módulos de portais. Para cada classe pode haver no máximo um portal. Um portal é encarregado de manejar a interação remota e transferência de dados entre a classe associada e ele. As declarações contidas em um portal identificam classes que podem ser referenciadas em chamadas remotas, além de identificar os métodos de uma classe que serão exportados. Um portal é associado automaticamente a uma instância de uma classe quando a referência a este objeto for exportada para fora do espaço de execução em que o objeto foi criado.

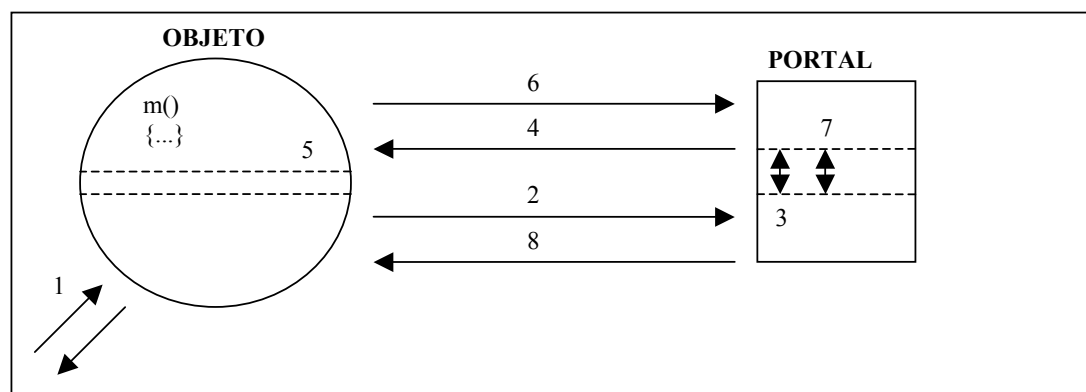


Figura 2.4 – Protocolo de um portal RIDL [LOP97]

O protocolo da Figura 2.4 é definido por [LOP97] através dos seguintes passos:

1. De algum espaço remoto ocorre uma invocação ao método *m* do objeto.
2. A requisição é encaminhada para o portal do objeto.
3. O método é processado de acordo com sua declaração no portal, onde os parâmetros são passados do espaço remoto para o espaço local de acordo com a especificação do portal.
4. A requisição é enviada ao objeto
5. O método *m* é executado
6. Quando a invocação retorna, o retorno é apresentado ao portal do objeto.
7. O resultado é processado de acordo com sua declaração de operação remota.
8. A invocação do método retorna e o resultado é passado ao espaço remoto.

O portal e o objeto se dividem com as linhas pontilhadas para representar os manipuladores local e remoto.

2.2.4 AspectJ

AspectJ [ASP03] é uma linguagem de aspectos de propósito geral que estende a linguagem *Java* para suporte a programação orientada a aspectos de maneira genérica. É uma ferramenta *open source*, possuindo suporte a ambientes integrados de desenvolvimento (*Emacs*, *Forté4J*, *NetBeans*, *JBuilder*). O projeto da linguagem é dirigido através da resposta dos usuários em relação ao ambiente.

É a ferramenta de suporte à programação orientada a aspectos mais conhecida e mais utilizada, é originada nas idéias do framework *D* [LOP97]. O combinador de aspectos utilizado por ela é genérica, possibilitando ao desenvolvedor especificar instruções em vários pontos de combinação. Alguns exemplos de pontos de combinação disponíveis são: a invocação e execução de métodos, inicialização de objetos, execução de construtores, tratamento de exceções, acesso e atribuição a atributos.

O combinador de aspectos do *AspectJ* é implementado como um pré-processador que recebe o programa de componentes e o programa de aspectos como entrada, gerando arquivos *Java* que podem ser compilados normalmente.

Em *AspectJ*, um aspecto normalmente define *pointcuts*, que são formados pela composição de pontos de combinação, através de combinadores lógicos. Para definir *pointcuts* utiliza-se construtores de *AspectJ* chamados designadores de *pointcut* (*pointcut designators*). Dentro dos aspectos temos ainda os *advices*, que são construções que definem o código adicional que será executado nos pontos de combinação. Existem três tipos de *advices*: *after*, *before* e *around*, que são executadas depois, antes e durante os pontos de combinação. Utilizando-se o *advice around* é necessário utilizar a função *proceed()* chamando o código original do ponto de combinação

Em *AspectJ* os aspectos são definidos por declarações que têm forma similar a declarações de uma classe. As declarações de aspectos podem incluir *pointcuts*, *advices*, além de outros tipos de declarações permitidas nas declarações de classes, como métodos e atributos. Ainda é possível utilizarmos os conceitos de herança impostos pelo conceito OO, sendo que os aspectos podem estar hierarquizados entre si.

2.2.5 AspectC

AspectC [COA01] é uma linguagem de aspectos de propósito geral que estende a linguagem C. É dito *AspectJ-like*, apesar de não oferecer nenhum tipo de suporte à programação orientada a objetos ou módulos explícitos.

O código de aspectos, conhecido como *advice*, interage com a funcionalidade básica nos limites de uma chamada a uma função e pode ser executado antes, depois ou durante a chamada. Os elementos centrais desta ferramenta têm como objetivo assinalar chamadas de funções particulares, acessando os parâmetros destas chamadas e anexar *advices* a elas. Como a linguagem C se trata de uma linguagem estática, o combinador do *AspectC* é estático.

Esta ferramenta, apesar de ser simples, é de grande utilidade devido à grande utilização da linguagem C em diversos tipos de aplicações. *AspectC* é ideal para o desenvolvimento de sistemas onde a eficiência é primordial, como na implementação de Sistemas Operacionais, trabalho que vem sendo desenvolvido por [COA01].

2.2.6 AspectC++

AspectC++ [ASC03] [GAL01] é uma linguagem de aspectos de propósito geral que estende a linguagem *C++* para suportar a orientação a aspectos. Nesta linguagem os pontos de combinação são pontos no código de componentes onde os aspectos podem interferir. Estes pontos podem se referir a código, tipos, objetos e fluxos de controle.

Os *pointcuts* são utilizados para identificar um conjunto de pontos de combinação. Eles são compostos de *pointcut designators* e uma série de operadores. A declaração de *advices* podem ser usados para especificar o trecho de código que deve ser executado nos pontos de combinação, especificado pelos *pointcuts*. Assim como no *AspectJ*, podemos identificar diferentes tipos de *advices*: *after*, *before* e *around*, que são executadas depois, antes e durante os pontos de combinação. Utilizando-se o *advice around* é necessário utilizar a função *proceed()* chamando o código original do ponto de combinação. Estes conceitos mostram a que a base para desenvolvimento de tal linguagem é o *AspectJ*.

Os aspectos em *AspectC++* implementam de forma modular as preocupações e são extensões do conceito de classes em *C++*. Os aspectos podem conter atributos e métodos, além de declarações de *advices*, é claro. Os conceitos de hierarquia também são mantidos para os aspectos, podendo um aspecto derivar de outro aspecto ou até de uma classe, só não é possível derivar uma classe de um aspecto.

A Figura 2.5 traz a arquitetura básica do compilador *AspectC++*. Primeiramente o código fonte escrito em *AspectC++* é analisado léxico, sintático e semanticamente. Logo se inicia o processo de planificação, onde os *pointcuts* são avaliados e os conjuntos de pontos de combinação são calculados. Um plano é criado para o combinador de aspectos (*Weaver*) contendo os pontos de combinação e as operações a serem realizadas em cada um deles. O combinador é responsável por transformar o plano em comandos concretos baseados na árvore sintática *C++*. Em seguida o manipulador realiza as trocas necessárias no código. A saída do manipulador é um código fonte *C++* com o código de aspectos combinado dentro dele.

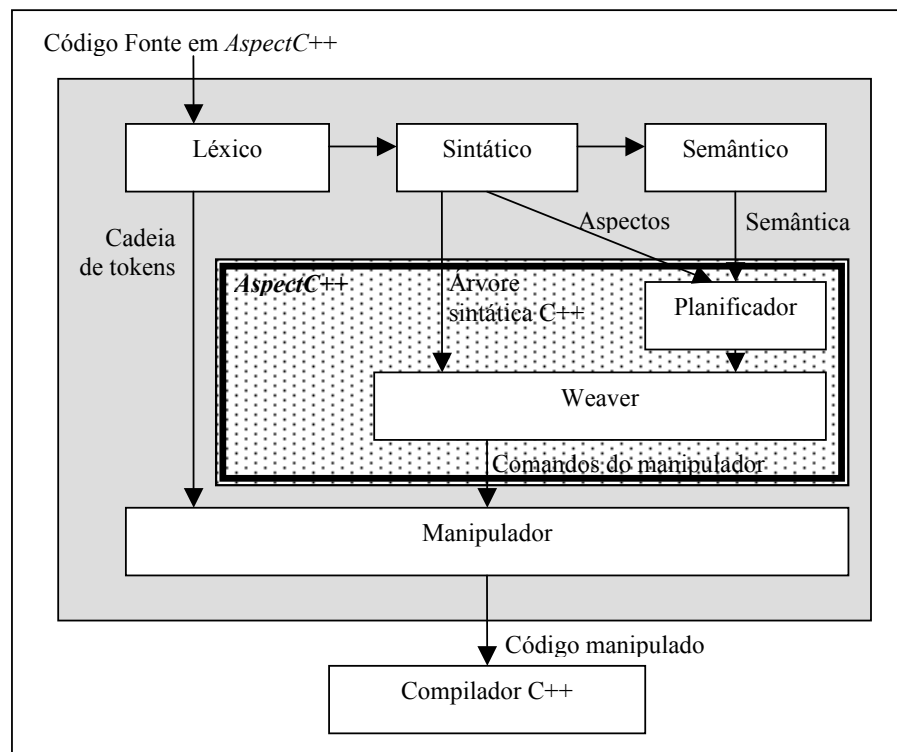


Figura 2.5 – Arquitetura do compilador *AspectC++* [GAL01]

Esta linguagem introduz importantes conceitos dentro do modelo de pontos de combinação, permitindo que estes atuem sobre classes, objetos e sobre o fluxo de controle.

2.2.7 *AspectS*

AspectS [ASS03] [HIR02] é uma linguagem de aspectos de propósito geral que estende a linguagem *Squeak / SmallTalk* para suportar a orientação a aspectos. Esta ferramenta utiliza o modelo da linguagem *AspectJ* e relacionando os aspectos e um ambiente dinâmico, empregando composição de meta-objetos.

Nesta linguagem os aspectos são implementados através de classes e suas instâncias atuam como um objeto, respeitando o princípio de uniformidade. Um aspecto pode conter um conjunto de *receptors*, *senders* ou classes *senders*. Estes objetos são adicionados ou removidos pelo código cliente e serão utilizados pelo processo de combinar em tempo de execução para determinar se o comportamento deve ser ativado ou não.

Em *Smalltalk*, a interação dos objetos está baseada na troca de mensagens. Sendo assim os pontos de combinação do *AspectS* são implementados através de envio de mensagens. Os objetos de *advice* associam fragmentos de código de um aspecto e seus respectivos pontos de combinação, que descrevem as tarefas a serem executadas pelo combinador de aspectos, que deve estes fragmentos dentro do sistema. Para representar estes fragmentos se utilizam blocos. Nesta linguagem um *advice* pode ser do tipo *AsBeforeAfterAdvice* (Antes e depois da chamada de um método), *AsHandlerAdvice* (Para tratar exceções) ou *AsAroundDevice* (Durante a chamada de um método).

AspectS utiliza o método de combinação dinâmico que transforma o sistema original de acordo com o que foi especificado nos aspectos. O processo de combinar acontece cada vez que uma instância de aspectos é instalada através do envio da mensagem *install* à respectiva instância do aspecto. Para reverter os efeitos de um aspecto, o aspecto deve ser desinstalado através do envio da mensagem *uninstall* à respectiva instância do aspecto. Este processo é chamado ‘descombinação’ (*unweaving*).

2.2.8 Comparativo

Na tabela a seguir se apresentam as principais características das linguagens apresentadas neste capítulo:

Linguagem	Linguagem de Componentes	Combinador de Aspectos	Propósito
<i>HyperJ</i>	<i>Java</i>	Dinâmico	Geral
<i>AOP/ST</i>	<i>VisualWorks/SmallTalk</i>	Dinâmico	Específico
<i>COOL</i>	Linguagens OO	Estático	Específico
<i>RIDL</i>	Linguagens OO	Estático	Específico
<i>AspectJ</i>	<i>Java</i>	Estático	Geral
<i>AspectC</i>	<i>C</i>	Estático	Geral
<i>AspectC++</i>	<i>C++</i>	Estático	Geral
<i>AspectS</i>	<i>Squeak/SmallTalk</i>	Dinâmico	Geral

Tabela 2.1 – Comparação de Linguagens e Ferramentas com suporte a aspectos

2.3 Considerações Finais

Este capítulo teve como objetivo apresentar os conceitos relacionados com a programação orientada a aspectos (AOP), além de algumas linguagens de aspectos existentes atualmente. Através da introdução dos conceitos básicos de AOP é possível compreender melhor o contexto em que este trabalho está inserido. A apresentação de conceitos básicos é imprescindível para a compreensão dos objetivos deste trabalho.

Após análise comparativa e pesquisa, foi escolhida uma das linguagens apresentadas afim de se implementar os estudos de caso. A linguagem escolhida foi a *AspectJ*, por ser uma ferramenta de maior popularidade e estabilidade, além de ser de propósito geral e ter como linguagem de componentes uma linguagem conhecida, o *Java*.

Os conceitos introduzidos e a linguagem de aspectos escolhida servirão como base para a definição de uma proposta de modelagem orientada a aspectos que respeite os conceitos AOP e a semântica do *AspectJ*.

Capítulo 3

Proposta de Modelagem Orientada a Aspectos

A maior parte dos trabalhos relacionados à AOP tratam o nível de implementação, porém grande preocupação atual está em outros níveis, como o de modelagem.

Atualmente não existem princípios de modelagem apropriados para modelar AOP em todas as linguagens com suporte a esta abordagem, devido à grande diferença existente entre as técnicas e linguagens existentes. Neste trabalho é feita uma proposta de modelagem para a linguagem *AspectJ*. Esta notação tem como objetivo facilitar a tarefa de modelar e compreender programas feitos em *AspectJ*. A proposta deste trabalho é estender a *Unified Modeling Language (UML)*, para o uso de AOP, como é especificada no *AspectJ*.

3.1 UML

A *UML* foi concebida por Grady Booch, James Rumbaugh e Ivar Jacobson, tendo sido aprovada pela *OMG (Object Management Group)* em novembro de 1997. *UML* é uma linguagem de modelagem para especificar, visualizar, construir e documentar os artefatos de um processo de software [ZAK02].

UML provê um grande número de conceitos e notações com o intuito de satisfazer os requisitos típicos da modelagem de sistemas. Para que não ocorram situações em que é necessária a introdução de conceitos e notações além dos predefinidos pela linguagem, a *UML* possui mecanismos que permitem estendê-la de forma consistente. Estes mecanismos fornecem meios para introduzir novos modelos com uma maior expressividade, definindo extensões específicas das linguagens de implementação ou específicas dos processos de desenvolvimento e associando informações semânticas aos elementos do modelo. Esses mecanismos representam extensões da própria linguagem de modo a alterar a estrutura e a semântica dos modelos criados

O mecanismo de extensibilidade da *UML* na verdade é um subpacote que especifica como elementos do modelo *UML* são personalizados e estendidos adicionando nova semântica através do uso de *stereotypes*, *constraints* e *tagged values*. Um conjunto coerente dessas extensões é chamado de *UML profile*.

Stereotype permite a classificação de um elemento de modelo estendido de acordo com um elemento de modelo base já existente na *UML* e a definição de novas restrições, além de uma nova representação gráfica [CAV02]. Todos os elementos estendidos do modelo que possuem um ou mais *stereotypes* terão os valores adicionais, restrições definidas pelos estereótipos, além dos atributos, associações e super classes que o elemento já possui na *UML*.

Constraints são regras de restrições bem definidas. Uma *constraint* permite que uma nova especificação semântica seja atribuída ao elemento de modelo. Tal especificação é escrita numa determinada linguagem de restrições, que pode ser mais ou menos formal. Uma *constraint* pode ser adicionada tanto a um elemento de modelo diretamente quanto ao *stereotype*, neste caso sendo aplicada a todos os elementos estendidos por esse *stereotype*.

Um *tagged value* permite associar arbitrariamente informação a qualquer elemento do modelo, sendo o seu significado intencionalmente específico dentro do contexto da sua utilização, podendo ser determinado pelo seu utilizador ou por convenções das ferramentas de suporte.

3.2 UML Profile para Modelar Aspectos em AspectJ

O que se propõe neste trabalho é uma modelagem simplificada para AOP, baseada na linguagem *AspectJ*. Para definir o *UML profile*, algumas definições das abstrações básicas contidas na linguagem *AspectJ* são seguidas.

3.2.1 Abstrações Básicas do AspectJ

Num primeiro momento vale lembrar que em *AspectJ*, os aspectos são definidos por declarações muito similares à definição de classes. Um aspecto pode conter atributos e operações, da mesma forma que uma classe. Além das definições básicas de uma classe,

um aspecto pode ainda conter *pointcuts* e *advices*. Os aspectos em *AspectJ* ainda herdam do modelo de classes os conceitos de herança.

Um *pointcut* é formado de vários pontos de combinação (*join points*), através de operadores lógicos. Abaixo um trecho de código contendo um *pointcut*:

```
pointcut Move():call (void Linha.setP1 (Point)) ||
                call (void Linha.setP2 (Point));
```

O *pointcut* acima contém dois pontos de combinação. Ele identifica as invocações dos métodos `SetP1` e `SetP2` de um objeto da classe `Linha`.

Um *advice* é uma construção que define um código adicional que deverá ser executado nos pontos de combinação.

```
after(): Move() { flag = true }
```

O *advice* acima define que a variável `flag` deverá ter o valor *true* atribuído a ela depois (*after*) da computação do *pointcut* `Move()`.

3.2.2 Modelando Aspectos em AspectJ

Os mecanismos de extensão da *UML* são baseados principalmente nos conceitos de *stereotypes*, que provêem um meio de classificar elementos de forma que eles se comportem como se fossem instâncias do novo meta-modelo [ALD01]. No presente trabalho, serão utilizados *stereotypes* para identificar aspectos, *pointcuts* e *advices*, na linguagem *UML*.

Classes

Primeiramente, será definida como ficará a estrutura da classe que terá o comportamento modificado pelos aspectos que serão inseridos no sistema. Para tal é proposto a criação de uma estrutura não definida nos mecanismos de extensão propostos pela *UML*, contendo

definições de atributos, métodos e aspectos relacionados àquela classe. Esses aspectos citados podem possuir pontos de combinação de qualquer natureza, desde que relacionados com algum elemento da classe. Pode ser relacionado com a invocação e execução de métodos, inicialização de objetos, execução de construtores, tratamento de exceções, acesso e atribuição a atributos. Estes últimos serão especificados posteriormente, na definição dos *pointcuts*. Na definição da classe apenas será definida uma ligação para o aspecto que, de alguma forma, interfere em seu comportamento.

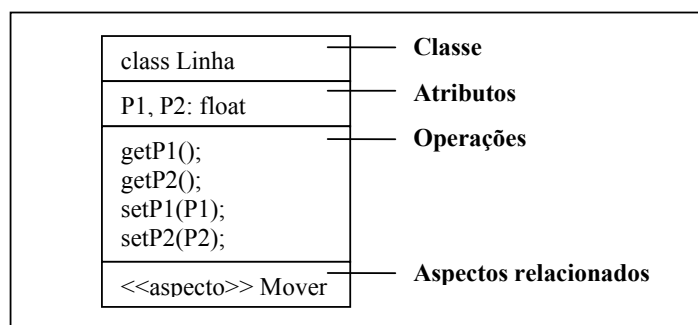


Figura 3.1 – Representando uma Classe com campo para o Aspecto

A Figura 3.1 mostra a estrutura de uma classe em *UML* com a adição do campo que contém os aspectos relacionados à classe, sem necessidade de utilizar relacionamentos entre aspectos e classes. A utilização de relacionamentos pode causar uma dificuldade de entendimento em sistemas de grande porte. Quando houverem muitas classes e muitos aspectos, mais de uma classe pode ser afetada por um determinado aspecto. Da mesma forma vários aspectos podem afetar uma classe. Este excesso de relacionamentos poderia levar a uma redução no entendimento do modelo, fugindo ao propósito de AOP, que busca uma maior modularidade e um entendimento mais fácil do sistema.

Aspectos

Como descrito anteriormente os aspectos em *AspectJ* são semelhantes às classes *Java*. Nesta proposta, então, um aspecto teria a representação de uma classe, sendo apenas decorado com o *stereotype* `<<aspecto>>`, como podemos observar na Figura 3.2.

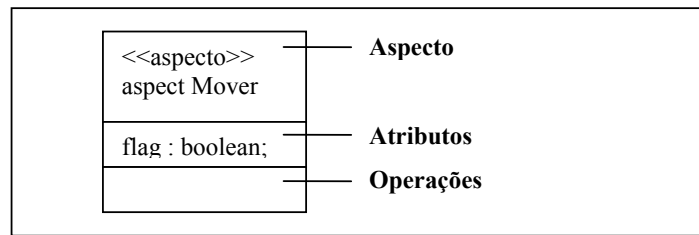


Figura 3.2 – Representando um Aspecto

Como pode-se perceber, não existem modificações consideráveis ou que possam comprometer o modelo nesta estrutura. A única alteração feita foi a inserção do *stereotype* <<aspect>> e a utilização da palavra reservada *aspect*, ao invés de *class*.

Agora surge a questão: onde ficariam os *pointcuts* e os *advices*? Esta pergunta é respondida com a criação de estrutura que contém as duas definições, de forma que estas se relacionem com o aspecto da qual fazem parte.

A primeira parte é a criação de uma estrutura que contenha as definições dos *pointcuts* e *advices*. Para tal será utilizado também o elemento classe do meta-modelo *UML*. Nesta modelagem cada *pointcut* será tratado como se fosse uma classe, sendo este apenas decorado com o *stereotype* <<pointcut>>. Como operações desses *pointcuts* são definidos os *advices* referentes a eles, de forma a separar estes conjuntos de pontos de combinação, prezando a modularidade na modelagem do sistema.

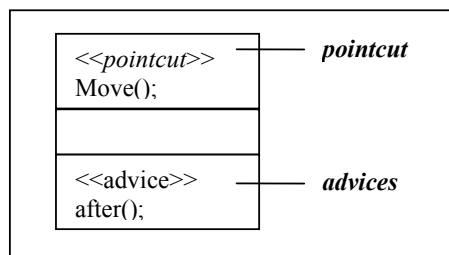


Figura 3.3 – Representando um *pointcut* e seus *advices*

A Figura 3.3 mostra como é representado um *pointcut* estendendo-se o elemento classe da *UML*. O entendimento se torna fácil, alterações, quando necessárias, são realizadas de maneira a não comprometer outras estruturas do modelo.

Ainda não houve qualquer forma de relacionamento entre o *pointcut* e o aspecto ao qual este faz parte. Esta questão foi resolvida através da utilização de *extends*. Os *extends* terão sua função um pouco alterada, pois, na modelagem apresentada, servirão para demonstrar quais *pointcuts* estão relacionados a um determinado aspecto. Pode-se visualizar a utilização dos *extends* na Figura 3.4.

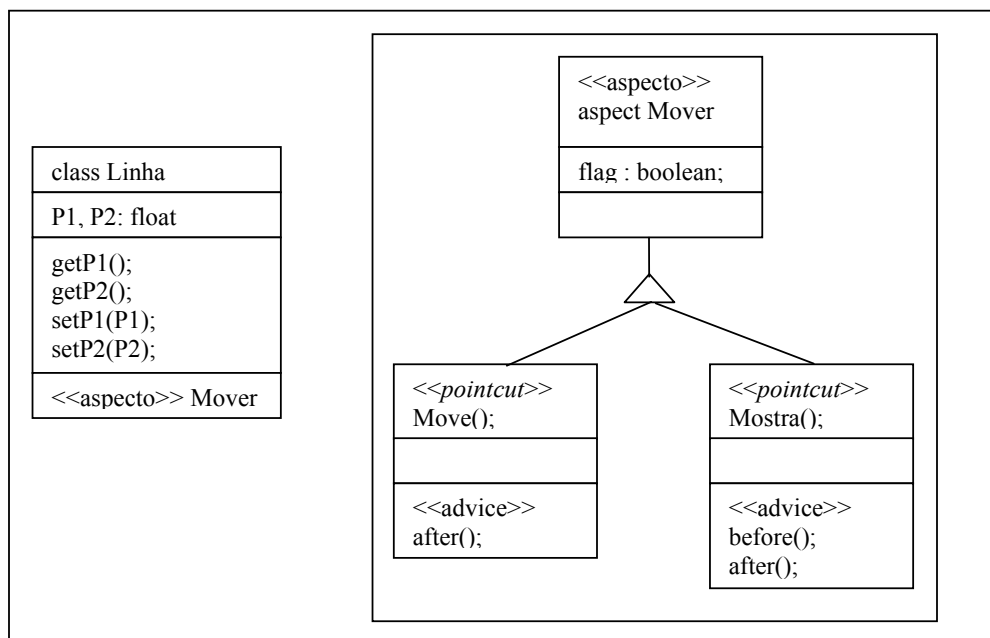


Figura 3.4 – Relacionando Classes, Aspectos e seus Pointcuts

Surge agora o problema com relação aos *extends*, uma vez que *AspectJ* oferece a possibilidade de se criar aspectos como sendo *extend* de outro. Isto foi resolvido criando-se uma espécie de “pacote”. Estes “pacotes” envolvem todo o aspecto, isto é, aspecto, atributos, operações, *pointcuts* e *advices*, de forma a demonstrar toda a estrutura daquele aspecto que seria estendido. O *extend* então seria criado a partir destes pacotes.

3.2.3 Trabalhos Relacionados

A busca por um padrão de modelagem para a programação orientada a aspectos tem sido muito pesquisada ultimamente. As principais propostas de modelos são as de Suzuki e Yamamoto [SUZ99], Siobhán Clarke [CLA02a] [CLA02b], Zakaria, Hosny e Zeid [ZAK02] e Stein, Hanenberg e Unland [STE02], as quais são descritas a seguir.

Abordagem de Suzuki e Yamamoto

Uma das primeiras propostas de estender *UML* afim de modelar programas baseados na orientação a aspectos. Em sua proposta uma meta-classe, chamada *aspect*, é criada e se relaciona com as classes que são afetadas pelos aspectos através de relacionamentos.

Esta abordagem contém um problema com respeito à especificação de *pointcuts* e *advices*. Não são propostos meios de se modelar estas estruturas utilizando *UML*, não ficando claro os efeitos que estes aspectos causam nas classes. Outro problema é o uso de relacionamentos, que formam modelos complexos.

Abordagem de Clarke

É a abordagem mais bem fundamentada e mais aceita por parte da comunidade que utiliza orientação a aspectos. Clarke estende a *UML* através de um novo conceito de modelagem, os chamados *composition patterns*. Estes *patterns* são na verdade *templates* de pacotes que utilizam classes e operações através de relacionamentos de composição especiais. *Composition patterns* são baseados numa notação especial para a *subject oriented programming*, chamada *subject oriented design model*.

Como pode-se perceber a abordagem de Clarke foi desenvolvida especificamente para modelar sistemas baseados na *subject oriented programming*, mas a autora busca demonstrar como os *composition patterns* podem ser utilizados para modelar programas orientados a aspectos. Clarke [CLA02a] mostra como implementar um sistema em *AspectJ*, utilizando uma modelagem que utiliza *composition patterns*. É possível entender a implementação a partir da abordagem, mas a maneira proposta não é compatível com a semântica do *AspectJ*. Um *advice* em *AspectJ* é executado dentro do escopo do *aspect* correspondente, enquanto utilizando a abstração proposta por Clarke é executado dentro do escopo da classe correspondente. Stein, Hanenberg e Unland [STE02] citam ainda como pontos fracos desta modelagem, a falta de suporte ao *crosscutting* recursivo e a impossibilidade de inserir-se definições de atributos e métodos dentro dos *composition patterns*.

Abordagem de Zakaria, Hosny e Zeid

Esta abordagem utiliza como base as estruturas da linguagem *AspectJ*. Os autores a utilizam a proposta de Aldawud, Elrad e Bader [ALD01], em que os mecanismos de extensão da *UML* são baseados nos *stereotypes*. Quando da introdução desta definição são citados exemplos para a criação de um *profile UML* para *Aspect-Oriented Software Development*, que são os aspectos, os *joinpoints*, os *advices*, os *pointcuts* e as associações.

A abordagem utiliza uma nova notação gráfica para representar aspectos e *pointcuts* como extensões do meta-modelo de classes *UML*. Os relacionamentos entre classes e aspectos são feitos através de relacionamentos de associação. Para tal é criado um conjunto de *tags* para identificar o tipo do relacionamento. As relações ficariam de forma que a classe se relacionaria com o aspecto através de um relacionamento de associação, a classe teria uma relação com o *pointcut*, decorado com a tag `<<has pointcut>>` e este mesmo *pointcut* teria relação com o aspecto através da tag `<<has pointcut>>`. Além destes ainda existe um tipo de relacionamento aspecto-aspecto, para especificar uma possível maior prioridade de um aspecto sobre o outro, para isto se utilizaria a tag `<<dominates>>`.

Esta proposta é muito bem fundamentada, e possui algumas facilidades para implementação em *AspectJ*, uma vez que traz todas as estruturas suportadas pela linguagem modeladas, mas peca em alguns sentidos. Ao introduzir novos ícones, pode trazer dificuldade aos desenvolvedores acostumados com a *UML*. Outro problema é o excesso de *tags* introduzidas por esta proposta, somente nas associações aspecto-classe, existem oito novas *tags*.

Como já citado na proposta de Suzuki e Yamamoto, os relacionamentos entre classes e aspectos podem aumentar a complexidade em sistemas de maior porte. No caso desta proposta, um problema ainda maior, pois é criado um relacionamento aspecto-classe, um relacionamento aspecto-*pointcut* e um relacionamento classe-*pointcut* para cada ponto de combinação, quando não for necessário um relacionamento aspecto-aspecto.

Abordagem de Stein, Hanenberg e Unland

Esta abordagem também é baseada nos conceitos introduzidos pelo *AspectJ*, sendo a que mais se aproxima de sua semântica. Nela é criada uma notação única para esta linguagem, tomando como base todos os elementos descritos na especificação da linguagem. Todos os novos elementos criados têm base nos *stereotypes*, estendendo os elementos classe da *UML*.

Para representar os aspectos é utilizado o *stereotype* `<<aspect>>` seguido do nome do aspecto. Para descrever certas diferenças que possam existir entre aspectos e classes são utilizadas algumas cláusulas, que dizem respeito principalmente aos mecanismos de herança e instanciação, trazendo, geralmente, o tipo de instanciação, a classe base e uma cláusula que diz se o aspecto pode os membros da classe base como uma *privileged friend*.

Dentro da extensão criada podemos encontrar ainda as estruturas básicas de uma classe, os atributos e as operações. Os atributos funcionariam normalmente, visto a possibilidade de um aspecto em *AspectJ* suportar a existência de atributos da mesma maneira que uma classe o faz. Dentro do espaço reservado para as operações encontramos as operações declaradas dentro do aspecto assim como os *pointcuts* e os *advices*.

Os *pointcuts* são decorados com o *stereotype* `<<pointcut>>` e são declarados como um método normal, exceto pela existência de restrições. A principal dessas restrições diz respeito ao ponto de combinação, dentro da classe base, ao qual se aplica o *pointcut*. Isto é feito afim de um maior entendimento com relação aos pontos em que o sistema é afetado por aquele *pointcut*.

Os *advices* são também decorados, mas com o *stereotype* `<<advice>>` e são declarados logo após os *pointcuts*. Estes possuem uma semântica mais simples, pois é muito próxima da semântica de um método *Java*. Com isso não foi necessária a introdução de nenhum tipo de restrição ou qualquer alteração além do *stereotype*.

Esta abordagem foi utilizada como base para a proposta deste trabalho, pois consegue se aproximar muito da semântica do *AspectJ*, trazendo para a fase de modelagem as

facilidades de modularidade da programação orientada a aspectos. Apesar disso, podem ser citados alguns pontos que não foram aceitos na criação da abordagem mostrada neste trabalho. A utilização de *pointcuts* e *advices* dentro da estrutura do aspecto pode causar uma certa dificuldade ao desenvolvedor. Esta dificuldade pode ser encontrada quando for necessária a alteração de algum *pointcut* ou *advice*, o que dificultaria a busca e alteraria a estrutura de todo o aspecto. São criadas algumas restrições de pouca importância quando da implementação do sistema.

Não é demonstrada como seria a relação classe-aspecto, mas, pode-se abstrair que estas não existem, uma vez que são criadas restrições junto aos *stereotypes* que mostram quais as classes relacionadas com os aspectos. Com isto pode-se apenas saber as classes afetadas pelo aspecto a partir do aspecto, e não o contrário, a partir das classes chegar aos aspectos que são combinados a elas.

3.3 Considerações Finais

Este capítulo traz a proposta de modelagem inserida nos objetivos deste trabalho. Para a concretização desta, foram analisadas algumas abordagens existentes. A proposta desenvolvida busca simplificar a modelagem de sistemas orientados a aspectos, excluindo-se relacionamentos classe – aspectos, que trazem muita complexidade a sistemas de grande porte. Para tal foram criados, na definição das classes, campos identificados com o *stereotype* <<aspecto>>, que servem como ligação entre a classe e os aspectos que interferem no seu comportamento.

A abordagem proposta introduz meios de modelar as estruturas do *AspectJ*, introduzindo novas estruturas que suportem os elementos encontrados na linguagem. Foram inseridas formas diferenciadas para lidar com *pointcuts*, *advices* e aspectos, além do suporte à herança entre aspectos.

Um problema relacionado à esta proposta é o não reconhecimento por parte da UML da estrutura criada para representar uma classe. Esta estrutura contém um campo extra para a definição dos aspectos que se relacionam com a classe. Apesar de algumas ferramentas permitirem a adição de campos à estrutura da classe, a UML não provê mecanismos para

fazê-lo. O que pode-se sugerir é a modificação da UML afim de permitir a utilização desta representação na modelagem de aspectos.

A proposta será utilizada nos estudos de caso apresentados no próximo capítulo, para observar o seu comportamento na implementação de sistemas com aspectos reais.

Capítulo 4

Estudo de Caso

Neste capítulo será apresentado um conjunto de estudos de caso simples, afim de ilustrar o uso da modelagem proposta, além de apresentar as implementações em *AspectJ*. A linguagem de aspectos *AspectJ* foi escolhida por ser uma extensão de uma linguagem conhecida, ser de propósito geral e por ser muito bem aceito pela comunidade que utiliza AOP. Será utilizado um módulo que provê suporte a aspectos dentro do *NetBeans* [NET03].

Primeiramente, será apresentado um exemplo simples afim de um melhor entendimento da linguagem e das estruturas da mesma. Será ilustrado o código fonte deste estudo de caso, seguido da proposta de modelagem. Em seguida será modelado um sistema simples que possui uma preocupação ortogonal (*crosscut concern*). Inicialmente será modelado para implementação Orientada a Objetos e implementado em *Java*.

Em seguida será identificado o código que está entrelaçado ou espalhado por todo o sistema e será encapsulado em um aspecto. Feito isto será utilizada a proposta de modelagem deste trabalho para o sistema exemplo. A modelagem será utilizada para implementar este sistema em *AspectJ*. Será feita, então, uma comparação com a implementação desta mesma aplicação sem utilização de aspectos.

4.1 Modelando Sistemas Orientados a Aspectos

4.1.1 Estudo de Caso: Hello World

Para manter a tradição da apresentação de uma nova linguagem e um novo paradigma, será apresentado um programa *HelloWorld* em *AspectJ*. Este *HelloWorld* terá algumas modificações, afim de que se possam introduzir aspectos. Primeiramente, deve-se implementar uma classe que contenha um método para imprimir uma mensagem, como pode ser observado na Figura 4.1

```

public class HelloWorld{
    public static void diz(String mensagem)
        System.out.println(mensagem);
    }
    public static void dizPessoa(String mensagem, String nome) {
        System.out.println(mensagem + ", " + nome);
    }
}

```

Figura 4.1 – Classe HelloWorld

Em seguida será introduzida uma saudação adicional. Antes do programa imprimir uma mensagem, deverá ser impressa uma saudação: “Olá!”. Após a impressão da mensagem, haverá uma resposta: “Muito Obrigado”. Para isto será apresentado um aspecto, denominado Saudação, na Figura 4.2.

```

public aspect Saudacao {
    pointcut Saudar() : call(* HelloWorld.diz*(..));
    before() : Saudar() {
        System.out.println("Olá!");
    }
    after() : Saudar() {
        System.out.println("Muito Obrigado!");
    }
}

```

Figura 4.2 – Aspecto Saudação

Tem-se então o aspecto Saudação, que possui um *pointcut* que captura todas as chamadas a métodos de qualquer tipo da classe *HelloWorld* que começam com “diz”, no exemplo proposto temos os métodos `diz()` e `dizPessoa()`. Foram então definidos dois *advices*: *after* e *before*, para que se possa imprimir as mensagens adicionadas, antes e depois do *pointcut* `Saudar()` ser atingido.

Agora será utilizada a proposta de modelagem deste trabalho para o caso deste HelloWorld. A Figura 4.3 traz o diagrama correspondente, contendo a classe HelloWorld e o pacote que contém o aspecto e seu *pointcut*.

Para exemplificar o funcionamento deste programa foi implementado o método `main()` dentro da classe *HelloWorld* e dentro dele se fez uma chamada ao método `diz()` :

```
public static void (String[] args){
    diz("Olá querido, tenha um bom dia!");
}
```

Quando fazemos a combinação dos códigos e compilamos temos como saída na tela a seguinte sequência:

```
> Olá!
> Olá Querido, tenha um Bom Dia!
> Muito Obrigado!
```

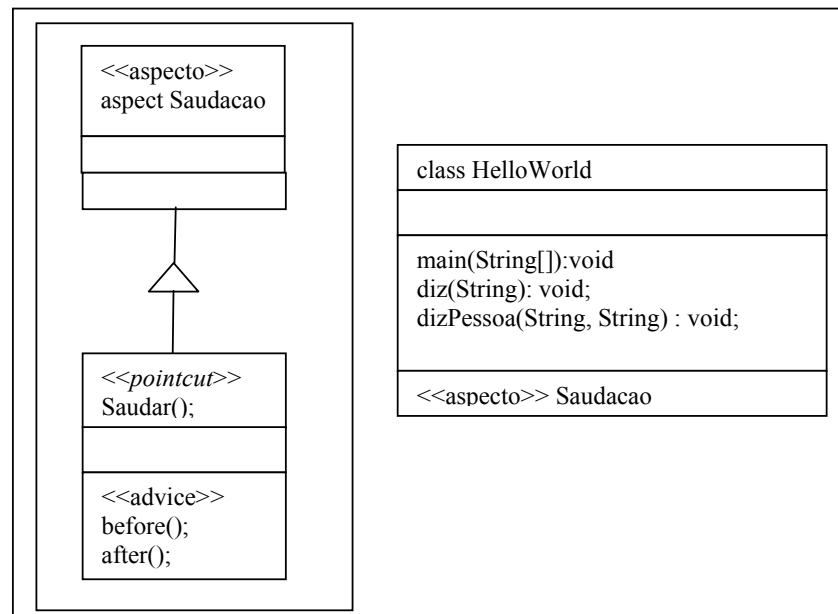


Figura 4.3 – Modelagem para o programa *HelloWorld* Orientado a Aspectos

```

public aspect SaudJaponesa {
    pointcut chamaPessoa (String pessoa):
        call(* HelloWorld.dizPessoa(String, String)) &&
        args(*, pessoa);
    void around(String pessoa) : chamaPessoa(pessoa) {
        proceed(pessoa + "-san");
    }
}

```

Figura 4.4 – Adicionando aspecto Saudação Japonesa

O aspecto ilustrado na Figura 4.4 traz o aspecto SaudJaponesa, que acrescenta “-san” depois do nome da pessoa, quando o método dizPessoa for chamado. Neste caso se implementou o *advice around*, para que o aspecto fosse chamado durante a execução do método, trocando o que seria o nome da pessoa pelo nome da pessoa acrescido de “-san”. Se o método main agora a seguinte chamada ao método dizPessoa(“Olá! Bom dia”, “Igor”), a saída seria visualizada da seguinte forma:

```

> Olá!
> Olá, Bom Dia Igor-san
> Muito Obrigado!

```

Com essa inserção a modelagem fica da seguinte forma:

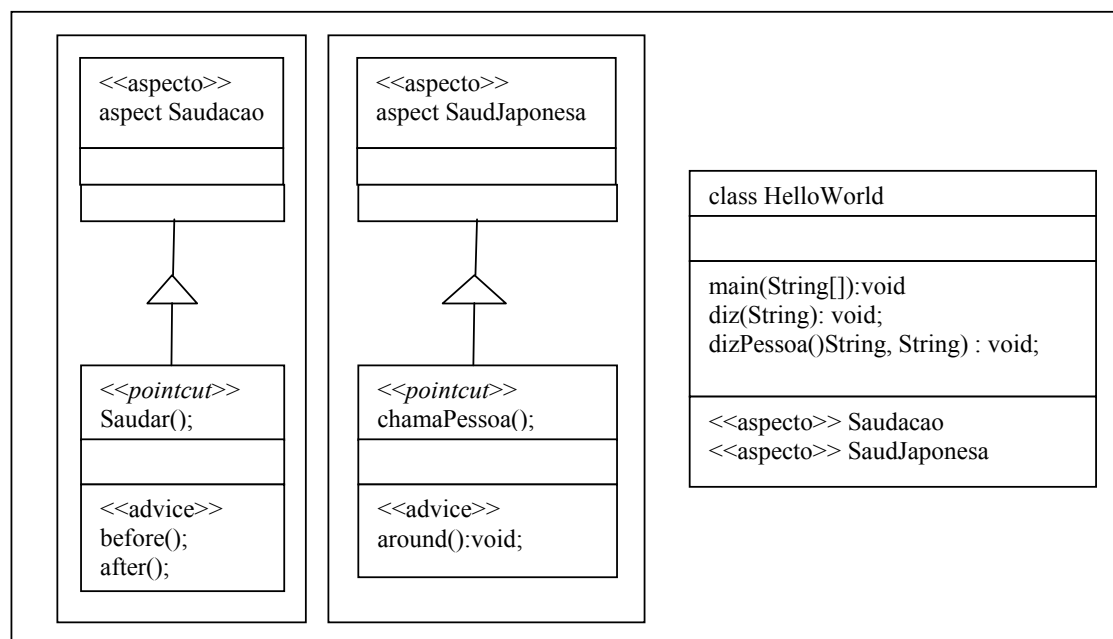


Figura 4.5 – Sistema com a inserção de outro aspecto

4.1.2 Estudo de Caso: *Tracing*

Implementar uma classe que provê os serviços de *tracing* é uma tarefa fácil: algumas funções, um *flag*, escolha de uma saída. A parte não trivial se encontra nas chamadas das funções de *tracing* nos momentos corretos. Em sistemas de grande porte, as chamadas às funções se encontrariam espalhadas por todo o código, retardando o andamento do projeto, já que estas chamadas devem ser retiradas do sistema antes de sua conclusão.

Utilizando-se aspectos é possível criar esta propriedade de *tracing* facilmente, pois ele pode ser visto como uma preocupação ortogonal que se espalha por todo o sistema, logo pode ser enxergado como um aspecto. Implementando como aspecto, pode-se inserir ou retirar as funcionalidade de *tracing*, quando se desejar, sem nenhum efeito na funcionalidade básica do sistema. Isto traz ganhos significativos em termos de tempo de desenvolvimento do sistema.

A Figura 4.6 traz a modelagem *UML* para implementação orientada a objetos de parte de um sistema que necessita ser rastreado. Para isso é implementada uma classe adicional *Trace* que possui métodos para fazê-lo. As chamadas destes métodos serão colocadas em seus lugares dentro do código que implementa as funcionalidades básicas. A implementação relacionada a este diagrama se encontra no Apêndice A.

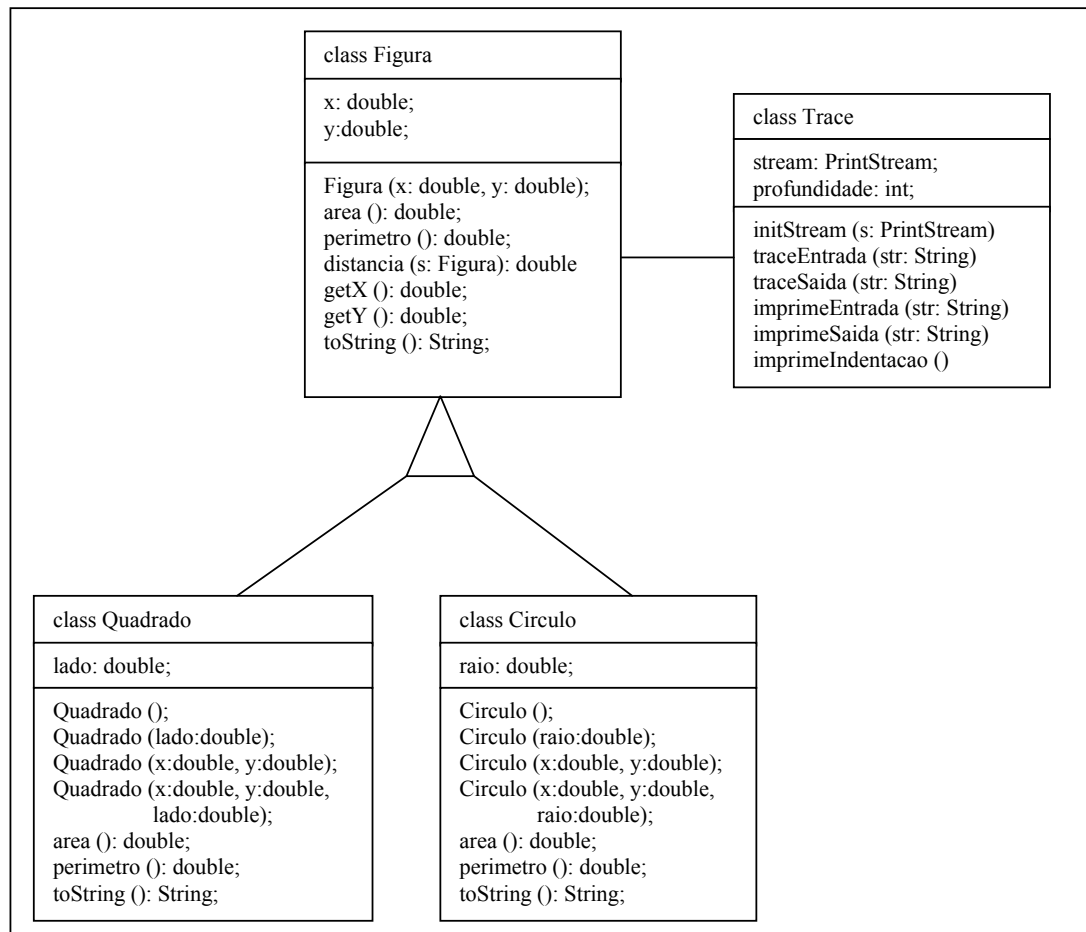


Figura 4.6 – Modelagem de um sistema Orientado a Objetos com *tracing*

Na figura 4.7 é possível verificar o entrelaçamento de código dentro da funcionalidade básica do sistema. Isto aumenta o custo de desenvolvimento, pois, como já dito, estas características devem ser excluídas quando da finalização do projeto, além de se ter de analisar cada método, afim de se ter o a chamada do *tracing* no local correto.

```

public abstract class Figura {

    protected double x, y;

    protected Figura(double x, double y) {
        this.x = x; this.y = y;
        Trace.traceEntrada(" Figura(double, double)");
        Trace.traceSaida(" Figura(double, double)");
    }

    public double getX() {
        Trace.traceEntrada(" double Figura.getX()");
        Trace.traceSaida(" double Figura.getX()");
        return x;
    }

    public double getY() {
        Trace.traceEntrada(" double Figura.getY()");
        Trace.traceSaida(" double Figura.getY()");
        return y;
    }

    public double distancia(Figura s) {
        Trace.traceEntrada(" double Figura.distancia(Figura)");
        double dx = Math.abs(s.getX() - x);
        double dy = Math.abs(s.getY() - y);
        Trace.traceSaida(" double Figura.distancia(Figura)");
        return Math.sqrt(dx*dx + dy*dy);
    }

    public String toString() {
        Trace.traceEntrada(" String Figura.toString()");
        Trace.traceSaida(" String Figura.toString()");
        return (" ["+String.valueOf(x)+", "+String.valueOf(y)+ "]");
    }

    public abstract double perimetro();
    public abstract double area();
}

```

Figura 4.7 – Classe Figura com código de *Tracing* entrelaçado

A Figura 4.8 apresenta a modelagem do mesmo sistema, agora com as funcionalidades de *tracing* utilizando aspectos. O *tracing* controlará toda a chamada de métodos e construtores das classes que possuem este aspecto.

Percebe-se na Figura 4.7 a existência de dois aspectos. Um primeiro chamado *Trace*, onde se encontram os atributos e métodos, além de alguns *pointcuts*. Este aspecto *Trace* contém os métodos responsáveis por imprimir de maneira correta os locais em que os métodos e construtores são chamados. Através dos métodos *traceEntrada* e *traceSaida*, são chamados os outros métodos que controlam a impressão e a indentação da impressão para rastrear corretamente as chamadas de métodos.

Existem nesse aspecto *Trace* três *pointcuts*. Um deles, *Classe*, é decorado com *abstract*, pois é definido em outro aspecto que estenda este. Os *pointcuts* *Método* e *Construtor* são responsáveis por mapear as chamadas de métodos e construtores dentro das classes mapeada pelo *pointcut* *Classe*, respectivamente. Cada um destes dois *pointcuts* possui os *advices* *before* e *after*, que são responsáveis por combinar seu código nos pontos de combinação especificados nos *pointcuts*. Especificamente os *advices* de *Método* devem inserir nos pontos de combinação uma chamada ao método *traceEntrada* antes da chamada dos métodos e uma chamada ao método *traceSaida* depois da execução dos métodos. Da mesma forma os *advices* de *Construtor* o fará com seus pontos de combinação.

O aspecto *TraceClasses* é uma extensão do aspecto *Trace*. Dentro dele é especificado o *pointcut* *Classes* e também o método *main* (), que é responsável por iniciar o *tracing*.

Para implementar as classes deve-se proceder da maneira convencional, descartando o com que está *stereotype* <<aspecto>>. A implementação dos aspectos é feita semelhantemente à uma classe. Por exemplo, para implementar o aspecto *Trace* deve-se criar os atributos e em seguida implementam-se os métodos. Ao terminar a implementação dos atributos e métodos declarados, deve-se iniciar a parte dos pontos de combinação. São criados então todos os *pointcuts*, especificando os pontos em que estes devem interferir. No caso do exemplo, o *pointcut* *Método* deve rastrear todas as chamadas de métodos dentro das classes especificadas no *pointcut* *Classe*. Após isso são declarados os *advices* correspondentes aos *pointcuts*, com o código que deve ser combinado ao código de componentes.

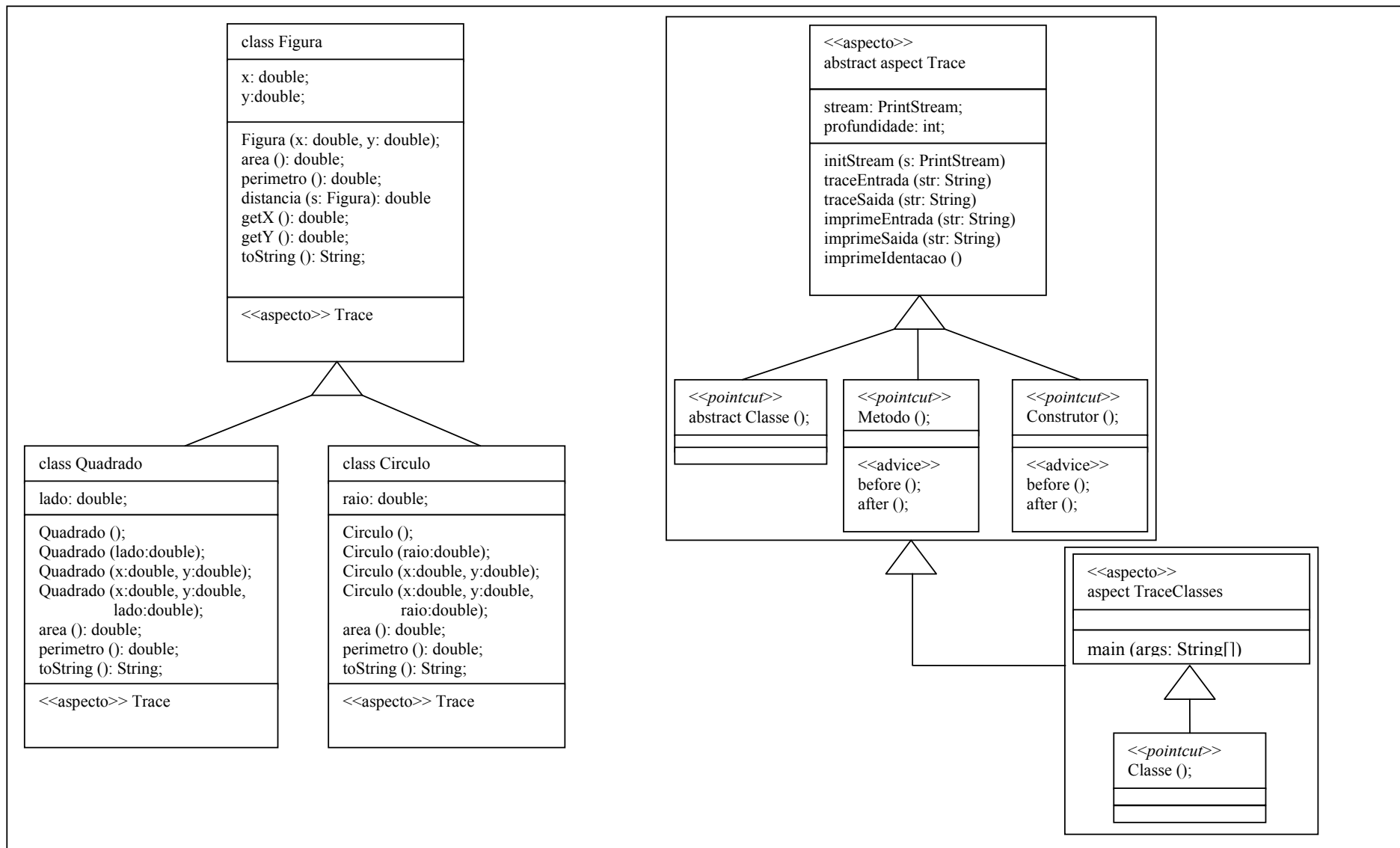


Figura 4.8 – Modelagem do sistema Orientado a Aspectos com Tracing

Na Figura 4.9 é mostrada a implementação do aspecto *Trace* segundo o que foi modelado. Pode-se perceber que a semântica da modelagem corresponde à semântica utilizada na implementação do sistema utilizando-se *AspectJ*, exceto pela definição dos pontos de combinação, que não são especificados, ficando a cargo do programador defini-los. Neste estudo de caso temos um exemplo de herança entre aspectos. O aspecto *TraceClasses* é uma extensão do aspecto *Trace* e para implementá-lo desta forma deve-se proceder como na implementação de uma classe. A Figura 4.10 mostra o aspecto *TraceClasses* implementado.

```
abstract aspect Trace {

    protected static PrintStream stream;
    protected static int profundidade;

    public static void initStream(PrintStream s) {
        stream = s;
    }
    //.....Implementação dos Outros Métodos .....

    /* PointCuts */

    abstract pointcut Classe();
    pointcut Construtor(): Classe() && execution(new(..));
    pointcut Metodo(): Classe() && execution(* *(..));

    /*advices*/

    before(): Construtor() {
        traceEntrada("" + thisJoinPointStaticPart.getSignature());
    }
    after(): Construtor() {
        traceSaida("" + thisJoinPointStaticPart.getSignature());
    }

    before(): Metodo() {
        traceEntrada("" + thisJoinPointStaticPart.getSignature());
    }
    after(): Metodo() {
        traceSaida("" + thisJoinPointStaticPart.getSignature());
    }

}
```

Figura 4.9 – Implementação de um aspecto modelado

```

public aspect TraceClasses extends Trace {

    pointcut Classe(): within(Figura) ||
                      within(Circulo) ||
                      within(Quadrado);

    public static void main(String[] args) {
        Trace.initStream(System.out);
        ExampleMain.main(args);
    }
}

```

Figura 4.10 – Aspecto *TraceClasse* como *extend* do aspecto *Trace*

Pode-se verificar ainda na implementação deste aspecto, a definição do *pointcut* *Classe*, que define quais classes serão afetadas pelo aspecto *Trace*.

Com a implementação destes aspectos, basta implementar as classes modeladas e criar um método de exemplo que instancie figuras, chame métodos das classes figura, quadrado e círculo. Com tudo implementado é necessário rodar o pré-processador do *AspectJ* e compilar usando o *Java*. O código completo da implementação *Java* e da implementação com *AspectJ* se encontram no Apêndice.

4.1.3 Uma análise comparativa

Para quantificar informalmente os benefícios da orientação a aspectos e da implementação utilizando *AspectJ* Asteasuain e Contreras [AST02] propõem uma métrica que leva em conta o tamanho dos arquivos das classes e dos aspectos. É descrito um fator de desenvolvimento que também será levado em conta. Este fator estabelece que para um sistema pequeno o tempo de desenvolvimento utilizando *Java* é de 10 horas/programador e usando *AspectJ* é de 2 horas/programador. Utilizando o exemplo implementado temos a seguinte tabela:

	Sem aspectos	Com aspectos
Figura. <i>Java</i>	1257 bytes	678 bytes
Circulo. <i>Java</i>	1470 bytes	690 bytes
Quadrado. <i>Java</i>	1452 bytes	676 bytes
Trace. <i>Java</i>	877 bytes	1494 bytes
Main. <i>Java</i>	737 bytes	678 bytes
TraceClass. <i>Java</i>	–	329 bytes
TOTAL	5739 bytes	4545 bytes

Tabela 4.1 – Comparativo AOP x OOP

Devemos então utilizar a fórmula sugerida por [AST02]:

$$BeneficioAOP = \frac{TamanhoSemAspectos * FatorDeDesenvolvimentoJava}{TamanhoComAspectos * FatorDeDesenvolvimentoAspectJ}$$

Trocando-se as variáveis pelos valores adquiridos temos:

$$BeneficioAOP = \frac{5739bytes * 10h/p}{4545bytes * 2h/p} = \frac{57390}{9090} = 6,32$$

Pode-se também quantificar a limpeza que se obtém no código através da seguinte métrica:

$$Limpeza = \frac{(TamanhoSemAspectos - TamanhoFuncionalidadeBasica) * 100}{TamanhoSemAspectos}$$

Aplicando ao exemplo, tem-se:

$$TamanhoFuncionalidadeBasica = 4545 \text{ bytes} - (678 + 329) \text{ bytes} = 3538 \text{ bytes}$$

$$Limpeza = \frac{(5739 - 3538)bytes * 100}{5739bytes} = 38,35$$

Pelos resultados pode-se verificar que a utilização de aspectos, neste caso, foi muito vantajosa. O Benefício AOP obtido 6,32, sendo que qualquer valor acima de 1 traduz um impacto positivo na mudança. O valor de 38,35% de limpeza conseguida pela utilização de AOP é outro ponto a ser considerado. Com isso pode-se verificar que a programação Orientada a Aspectos pode trazer grandes vantagens, em termos de custo com tempo, espaço e limpeza de código.

4.2 Considerações Finais

Neste capítulo aplicou-se a modelagem proposta neste trabalho a estudos de caso. Pode-se concluir que a abordagem se comportou bem tratando-se de sistemas de pequeno porte. Uma grande falha encontrada deve-se ao fato da não definição dos pontos de combinação dentro dos *pointcuts*. Isto pode causar incoerência dentro do processo de desenvolvimento do software. Por exemplo, durante a modelagem, um engenheiro pode imaginar que um

determinado *pointcut* atue na chamada de alguns métodos. Já na implementação o programador pode visualizar este *pointcut* atuando sobre a chamada de outros métodos.

De forma geral a técnica de modelagem proposta se mostrou correta para os sistemas propostos. A implementação em *AspectJ* a partir do modelo criado se tornou uma tarefa fácil.

Ao final do capítulo foi apresentada uma métrica baseada exclusivamente na implementação do sistema. Esta métrica demonstrou, para o estudo de caso apresentado, que a implementação utilizando AOP é mais benéfica e resulta num código muito mais limpo.

Capítulo 5

Conclusões

O paradigma AOP está surgindo como um paradigma promissor, pois elimina algumas falhas que as técnicas procedimental e orientada a objetos possuem. Ainda há muito o que se pesquisar nesta área, pois o que existe hoje são várias linguagens que usam aspectos de maneiras diferentes, e que se utilizam de linguagens de componentes existentes para combinar seu código. Já existem também muitas propostas de técnicas de modelagem orientada a aspectos, que se baseiam em conceitos e linguagens específicas. Surge aí um amplo campo de pesquisas para a criação de uma técnica de modelagem que respeite os princípios introduzidos pela AOP e que não se baseiem em uma linguagem específica.

A orientação a objetos só tem a ganhar com o crescimento da orientação a aspectos, pois essa eliminará grande parte de suas falhas, diminuindo a complexidade de implementação e aumentando a legibilidade e facilidade de manutenção. Haverá uma grande redução com relação ao problema de espalhamento de código, o que facilitará a modularização e o desenvolvimento de componentes mais facilmente, uma vez que AOP diminui muito o acoplamento. Com esta diminuição fica muito fácil inserir ou retirar aspectos em um sistema. Tudo isto leva a uma maior reutilizabilidade, tanto de classes como de aspectos. AOP peca quando tratamos da violação de encapsulamento, uma vez que um aspecto tem capacidade de interferir em métodos e classes privadas.

Este trabalho introduziu uma proposta de modelagem que suporte a AOP, baseando-se na semântica do *AspectJ*, bem como utilizou-se desta proposta para implementar estudos de caso. Para sistemas de pequeno porte como os demonstrados, a técnica de modelagem se mostrou correta e de fácil utilização. Com ela a tarefa de se modelar um aspecto ficou muito mais próxima da implementação dos mesmos, facilitando o trabalho do programador. A proposta mostra claramente o desacoplamento existente entre classes e aspectos, sem a utilização de relacionamentos entre estes. Demonstrou-se a facilidade de se inserir um aspecto em um sistema já estável, visto que isto não afeta o código de componentes aparentemente.

A abordagem proposta introduz meios de modelar as estruturas do *AspectJ*, inclusive inserindo o conceito de herança, que também é possível entre aspectos. Outro ponto a ser considerado é a facilidade de se lidar com os *pointcuts*, uma vez que estes são modelados através de uma estrutura, que se associa ao aspecto através de *extends*.

Foram encontradas algumas falhas na proposta, sendo a principal a falta da especificação dos pontos de combinação contidos em cada *pointcut*, o que pode causar confusão na implementação, pois estes pontos serão definidos pelo programador, que pode interpretar de maneira diferente o que o Engenheiro de Modelagem queria passar. Para implementação de sistemas simples não foram encontrados problemas. Mas, com certeza, a outros olhos, a modelagem pode evoluir de maneira a representar os aspectos de forma mais fiel e mais próxima da realidade.

No estudo de caso pudemos observar a maior complexidade em um código que possua sua funcionalidade básica entrelaçada com preocupações ortogonais (*crosscutting concerns*). Verificamos a diminuição da mão-de-obra ao implementarmos um sistema com *tracing* utilizando-se *AspectJ*, lembrando que este aspecto é, na grande maioria dos casos, utilizado apenas durante o desenvolvimento do software, devendo ser retirado para utilização por parte do usuário final. Houve uma redução significativa no tamanho do final do código fonte implementado manual. Ressaltamos que o código produzido pela combinação do código de aspectos e código de componentes, com a utilização do pré-processador, no entanto, é maior que o código orientado a objetos implementado.

Referências Bibliográficas

- [ALD01] ALDAWUD, Omar, ELRAD, Tzilla and BADER, Atef, "A *UML* Profile for Aspect Oriented Modeling", *OOPSLA 2001 - Workshop on Advanced Separation of Concerns in Object-Oriented Systems*, Tampa Bay, Florida, Outubro, 2001
- [ASC03] ASPECTC++, AspectC++ Home Page. <http://www.AspectC.org>, 2003.
- [ASP03] ASPECTJ, AspectJ Home Page. <http://www.AspectJ.org>, 2003.
- [ASS03] ASPECTS, AspectS Home Page <http://www.prakinf.tu-ilmenau.de/~hirsch/Projects/Squeak/aspects/AspectS.html>, 2003.
- [AST02] ASTEASUAIN, Fernando, CONTRERAS, Bernardo Ezequiel, "Programación Orientada a Aspectos – Análisis del Paradigma", *Tesis de Licenciatura, Departamento de Ciencias e Ingeniería de la Computación – Universidad Nacional del Sur, Buenos Aires – Argentina – Outubro de 2002*
- [CAV02] CAVALCANTI, Marcus R., MACHADO, Maria L., "Uma comparação na extensibilidade de Meta-Modelos", *Mestrado IM/NCE – UFRJ, Rio de Janeiro, 2002*
- [CLA01] CLARKE, Siobhán, WALKER, Robert J. "Composition Patterns: An Approach to Designing Reusable aspects", *Proceedings of ICSE '01*, Toronto, Canada, Maio 2001, ACM, 5-14
- [CLA02a] CLARKE, Siobhán, WALKER, Robert J., "Towards a Standard Design Language for AOSD", *Proceedings of the First International Conference on Aspect-Oriented Software Development*, Enschede, The Netherlands, 22 - 26 April, 2002.
- [CLA02b] CLARKE, Siobhán, "Extending standard *UML* with model composition semantics", *Science of Computer Programming*, Volume 44, pag. 71-100. Elsevier Science, Julho 2002

- [COA01] COADY, Yvonne, KICKZALES, Gregor, FEELEY, Mike, SMOLYN, Greg, “Using *AspectC* to improve the modularity of path-specific customization in operating system code”, *Proceedings of Joint ESEC and FSE-9*, 2001.
- [CZA00] CZARNECKI, Krzysztof, EISENECKER, Ulrich, *Generative Programming: Methods, Tools, and Applications*, Addison-Wesley, 2000.
- [ELR01a] ELRAD, Tzilla, FILMAN, Robert E., BADER, Atef, “Aspect-Oriented Programming”, *Communications of the ACM*, Vol. 44, Outubro 2001.
- [ELR01b] ELRAD, Tzilla, AKSIT, Mehmet, KICZALES, Gregor, LIEBERHERR, Karl, OSSHER, Harold, “Discussing *AspectS* of AOP”, *Communications of the ACM*, Vol. 44, Outubro 2001.
- [FER02] VIEIRA, Fernanda, FONTANA, Heitor, BRAGA, Marcelo. “Aspect Oriented Programming (AOP) - Uma visão geral da programação orientada a aspectos. Usando *AspectP*”, UFSC, 2002
- [GAL01] GAL, Andreas, SCHRÖDER-PREIKSCHAT, Wolfgang, SPINCZYK, Olaf, “*AspectC++*: Language Proposal and Prototype Implementation”, *OOPSLA 2001-Workshop on Advanced Separation of Concerns in Object-Oriented Systems*, Tampa Bay, Florida, October 14th, 2001
- [HIR02] HIRSCHFELD, Robert, “Aspect-Oriented Programming with Aspects” *Proceedings of Net.ObjectDays (NODe)*, Erfurt, October 2002
- [HYP03] Hyperspace web site, <http://www.research.ibm.com/hyperspace>.
- [KIC97] KICKZALES, Gregor, IRWIN, John, LAMPING, John, MENDHEKAR, Anurag, MAEDA, Chris, LOPES, Cristina Videira, LOINGTIER, Jean-Marc. “Aspect-Oriented Programming”, *ECOOP’97*, Finland: Springer-Verlag, 1997.

- [**LOP97**] LOPES, Cristina Isabel Videira, “D: a language framework for distributed programming”, *Ph.D. Thesis, Northeast University*, 1997.
- [**MUR01**] MURPHY, Gail C., BANIASSAD, Elisa L. A., ROBILLARD, Martin P., LAI, Albert, KERSTEN, Mik A., “Does Aspect-Oriented Programming Work?”, *Communications of the ACM*, Vol. 44, Outubro 2001
- [**NET03**] NETBEANS, NetBeans Home Page, <http://www.netbeans.org>, 2003
- [**OSS99**] OSSHER, Harold and TARR, Peri, “Using subject-oriented programming to overcome common problems in object-oriented software development/evolution”, In *International Conference on Software Engineering, ICSE'99*, pages 698–688. ACM, 1999
- [**OSS01**] OSSHER, Harold, TARR, Peri, “*Multi-Dimensional Separation of Concerns and the Hyperspace Approach*”, Proceedings of the Symposium on Software Architectures and Component Technology: The state of the Art in Software Development. Kluwer, 2001.
- [**PAR72**] PARNAS, David Lorge, “On the Criteria to be Used in decomposing Systems into Modules”, *Communications of the ACM*, Vol. 15(2), 1972
- [**PIV01**] PIVETA, Eduardo Kessler, ZANCANELLA, Luiz Carlos, “Um Modelo de Suporte à Programação Orientada a Aspectos”, *Dissertação de Mestrado, INE – Universidade Federal de Santa Catarina, Florianópolis –SC*, 2001
- [**SOA02**] SOARES, Sérgio, BORBA, Paulo, “AspectJ - Programação orientada a aspectos em Java”, *SBLP2002 – Simpósio Brasileiro de Linguagem de Programação*, Rio de Janeiro – RJ, 2002.
- [**STE02**] STEIN, Dominik, HANENBERG, Stefan, UNLAND, Rainer, “A UML-based Aspect-Oriented Design Notation For AspectJ”, *Proceedings of the First International*

Conference on Aspect-Oriented Software Development, Enschede, The Netherlands, 22--26 April, 2002.

[SUZ99] SUZUKI, Junichi, YAMAMOTO, Yoshikazu “Extending *UML* with Aspects: Aspect Support in the Design Phase”, *Proceedings of AOP Workshop at ECOOP '99*, Lisboa, Portugal, Junho 1999.

[ZAK02] ZAKARIA, Aida Atef, HOSNY, Hoda , ZEID, Amir, “A *UML* Extension for Modeling Aspect-Oriented Systems”, *Aspect-Oriented Modeling with AOP as part of the AOSD*, Enschede, The Netherlands – Abril de 2002

Bibliografia

- [AOS03] AOSD, **AOSD Home Page**. <http://www.aosd.net>, 2003.
- [ARA02] ARAÚJO, João, MOREIRA, Ana, BRITO, Isabel e RASHID, Awais, "Aspect - Oriented Requirements with *UML*", *Aspect-Oriented Modeling with AOP as part of the AOSD*, Enschede, The Netherlands – Abril de 2002.
- [CHA01] CHAVEZ, Cristina, LUCENA, Carlos, “Design Support for Aspect-Oriented Software Development”, *Doctoral Symposium and Poster Session at OOPSLA'2001*, Tampa Bay, Florida, USA, 14 – 18 de Outubro, 2001.
- [CHA02] CHAVEZ, Cristina, LUCENA, Carlos, “A Metamodel for Aspect-Oriented Modeling”, *Proceedings of the First International Conference on Aspect-Oriented Software Development*, Enschede, The Netherlands, 22--26 April, 2002.
- [CON02] CONSTANTINIDES, Constantinos A., SKOTINIOTIS, Therapon, “Reasoning About a Classification of Crosscutting Concerns in Object-Oriented Systems”, *Second Workshop on Aspect-Oriented Software Development*, Bonn, Alemanha, 21 – 22 de Fevereiro, 2002
- [HAN03] HANNEMAN, John, homepage, “Aspect-Oriented Design Pattern Implementations”, <http://www.cs.ubc.ca/~jan/AODPs/>
- [JEZ02] JÉZÉQUEL, Jean-Marc, PLOUZEAU, Noël, WEIS, Torben, GEIHS, Kurt, "From Contracts to Aspects in *UML* Designs", *Proceedings of the First International Conference on Aspect-Oriented Software Development*, Enschede, The Netherlands, 22-26 April, 2002.
- [KAN02] KANDÉ, Mohamed Mancona, KIENZLE, Jörg, STROHMEIER, Alfred, "From AOP to *UML* - A Bottom-Up Approach", *Aspect-Oriented Modeling with AOP as part of the AOSD*, Enschede, The Netherlands – Abril de 2002.

[LIP99] LIPPERT, Martin, LOPES, Cristina Videira, “A Study on Exception Detection and Handling Using Aspect-Oriented Programming”, *Xerox Palo Alto Research Center Technical Report P9910229 CSL-99-1*, Dezembro 1999.

Apêndice A

Código Fonte Tracing – Java

A.1 Classe Figura.Java

```
// Figura.Java → Define as funcionalidades básicas das Figuras
public abstract class Figura {

    //Coordenadas da figura
    protected double x, y;

    protected Figura(double x, double y) {
        this.x = x; this.y = y;
        Trace.traceEntrada(" Figura(double, double)");
        Trace.traceSaida(" Figura(double, double)");
    }

    // Retorna a coordenada X da figura
    public double getX() {
        Trace.traceEntrada(" double Figura.getX()");
        Trace.traceSaida(" double Figura.getX()");
        return x;
    }

    // Retorna a coordenada Y da figura
    public double getY() {
        Trace.traceEntrada(" double Figura.getY()");
        Trace.traceSaida(" double Figura.getY()");
        return y;
    }

    // Retorna a distancia entre a figura e outra passada por parametro
    public double distancia(Figura s) {
        Trace.traceEntrada(" double Figura.distancia(Figura)");
        double dx = Math.abs(s.getX() - x);
        double dy = Math.abs(s.getY() - y);
        Trace.traceSaida(" double Figura.distancia(Figura)");
        return Math.sqrt(dx*dx + dy*dy); //Faz Pitagoras
    }

    //Retorna o perimetro/area da figura → definidas nas subclasses
    public abstract double perimetro();
    public abstract double area();

    //Retorna uma string com informacoes....
    public String toString() {
        Trace.traceEntrada(" String Figura.toString()");
        Trace.traceSaida(" String Figura.toString()");
        return (" [" + String.valueOf(x)+ ", "+ String.valueOf(y)+ "] ");
    }
}
```

A.2 Classe Quadrado.Java

```
// Quadrado.Java → Define as funcionalidades básicas do quadrado
public class Quadrado extends Figura {
    protected double lado;    //tamanho do lado do quadrado

    //Os quatro construtores
    public Quadrado(double x, double y, double lado) {
        super(x, y); this.lado = lado;
        Trace.traceEntrada(" Quadrado(double, double, double)");
        Trace.traceSaida(" Quadrado(double, double, double)");
    }

    public Quadrado(double x, double y) {
        this(x, y, 1.0);
        Trace.traceEntrada(" Quadrado(double, double)");
        Trace.traceSaida(" Quadrado(double, double)");
    }

    public Quadrado(double lado) {
        this(0.0, 0.0, lado);
        Trace.traceEntrada(" Quadrado(double)");
        Trace.traceSaida(" Quadrado(double)");
    }

    public Quadrado() {
        this(0.0, 0.0, 1.0);
        Trace.traceEntrada(" Quadrado()");
        Trace.traceSaida(" Quadrado()");
    }

    //Retorna o perimetro do quadrado
    public double perimetro() {
        Trace.traceEntrada(" double Quadrado.perimetro()");
        Trace.traceSaida(" double Quadrado.perimetro()");
        return 4 * lado;
    }

    //Retorna a area do quadrado
    public double area() {
        Trace.traceEntrada(" double Quadrado.area()");
        Trace.traceSaida(" double Quadrado.area()");
        return lado*lado;
    }

    // Adiciona informações do quadrado ao método da Superclasse.
    public String toString() {
        Trace.traceEntrada(" String Quadrado.toString()");
        Trace.traceSaida(" String Quadrado.toString()");
        return ("Tamanho do Lado = " + String.valueOf(lado) +
super.toString());
    }
}
```

A.3 Classe Circulo.Java

```
// Circulo.Java → Define as funcionalidades básicas do circulo

public class Circulo extends Figura {
    protected double raio;

    //Os quatro construtores

    public Circulo(double x, double y, double raio) {
        super(x, y);
        this.raio = raio;
        Trace.traceEntrada(" Circulo(double, double, double)");
        Trace.traceSaida(" Circulo(double, double, double)");
    }

    public Circulo(double x, double y) {
        this(x, y, 1.0);
        Trace.traceEntrada(" Circulo(double, double)");
        Trace.traceSaida(" Circulo(double, double)");
    }

    public Circulo(double raio) {
        this(0.0, 0.0, raio);
        Trace.traceEntrada(" Circulo(double)");
        Trace.traceSaida(" Circulo(double)");
    }

    public Circulo() {
        this(0.0, 0.0, 1.0);
        Trace.traceEntrada(" Circulo()");
        Trace.traceSaida(" Circulo()");
    }

    //Retorna o perimetro do circulo
    public double perimetro() {
        Trace.traceEntrada(" double Circulo.perimetro()");
        Trace.traceSaida(" double Circulo.perimetro()");
        return 2 * Math.PI * raio;
    }

    //Retorna a area do circulo
    public double area() {
        Trace.traceEntrada(" double Circulo.area()");
        Trace.traceSaida(" double Circulo.area()");
        return Math.PI * raio*raio;
    }

    // Adiciona informações do circulo ao método da Superclasse.
    public String toString() {
        Trace.traceEntrada(" String Circulo.toString()");
        Trace.traceSaida(" String Circulo.toString()");
        return ("Raio do Circulo = " + String.valueOf(raio) +
super.toString());
    }
}
```

A.4 Classe Trace.Java

```
import Java.io.PrintStream;

//Trace.Java → Funcionalidades de imprimir as mensagens do tracing

public class Trace {

    protected static PrintStream stream = null;
    protected static int profundidade = 0; //nivel de indentacao

    //inicializa o stream
    public static void initStream(PrintStream s) {
        stream = s;
    }

    //imprime uma mensagem de entrada, deve ser chamado na entrada do
    //bloco a ser rastreado
    public static void traceEntrada(String str) {
        profundidade++;
        imprimeEntrada(str);
    }

    //imprime uma mensagem de saida, deve ser chamado na saida do
    //bloco a ser rastreado
    public static void traceSaida(String str) {
        imprimeSaida(str);
        profundidade--;
    }

    //string a ser impressa na entrada
    private static void imprimeEntrada(String str) {
        imprimeIndentacao();
        stream.println("--> " + str);
    }

    //string a ser impressa na saida
    private static void imprimeSaida(String str) {
        imprimeIndentacao();
        stream.println("<-- " + str);
    }

    private static void imprimeIndentacao() {
        for (int i = 0; i < profundidade; i++)
            stream.print(" ");
    }
}
```

A.5 Classe Exemplo.Java

```
//Classe Exemplo com método main para testar o tracing

public class Exemplo {

    public static void main(String[] args) {

        Trace.initStream(System.out);

        Circulo c1 = new Circulo(2.0, 3.0, 2.0);

        Circulo c2 = new Circulo(4.0);

        Quadrado q1 = new Quadrado(1.0, 2.0);

        System.out.println("---\nc1.perimetro() = " + c1.perimetro());
        System.out.println("---\nc1.area() = " + c1.area());

        System.out.println("---\nq1.perimetro() = " + q1.perimetro());
        System.out.println("---\nq1.area() = " + q1.area());

        System.out.println("---\nc2.distancia(c1) = " + c2.distancia(c1));
        System.out.println("---\nq1.distancia(c1) = " + q1.distancia(c1));

        System.out.println("---\nc1.toString(): " + c1.toString());
        System.out.println("---\nc2.toString(): " + c2.toString());
        System.out.println("---\nq1.toString(): " + q1.toString());
    }
}
```

Apêndice B

Código Fonte Tracing – *AspectJ*

B.1 Classe *Figura.java*

```
// Figura.java → Define as funcionalidades básicas das Figuras
public abstract class Figura {

    //Coordenadas da figura
    protected double x, y;

    protected Figura(double x, double y) {
        this.x = x; this.y = y;
    }

    // Retorna a coordenada X da figura
    public double getX() {
        return x;
    }

    // Retorna a coordenada X da figura
    public double getY() {
        return y;
    }

    // Retorna a distancia entre a figura e outra passada por parametro
    public double distancia(Figura s) {
        double dx = Math.abs(s.getX() - x);
        double dy = Math.abs(s.getY() - y);
        return Math.sqrt(dx*dx + dy*dy); //Faz Pitagoras
    }

    //Retorna o perimetro/area da figura → definidas nas subclasses
    public abstract double perimetro();
    public abstract double area();

    //Retorna uma string com informacoes....
    public String toString() {
        return (" [" + String.valueOf(x)+ ", "+ String.valueOf(y)+ "] ");
    }
}
```

B.2 Classe Quadrado.Java

```
// Quadrado.Java → Define as funcionalidades básicas do quadrado

public class Quadrado extends Figura {

    protected double lado;    //tamanho do lado do quadrado

    //Os quatro construtores
    public Quadrado(double x, double y, double lado) {
        super(x, y); this.lado = lado;
    }

    public Quadrado(double x, double y) {
        this(x, y, 1.0);
    }

    public Quadrado(double lado) {
        this(0.0, 0.0, lado);
    }

    public Quadrado() {
        this(0.0, 0.0, 1.0);
    }

    //Retorna o perimetro do quadrado
    public double perimetro() {
        return 4 * lado;
    }

    //Retorna a area do quadrado
    public double area() {
        return lado*lado;
    }

    // Adiciona informações do quadrado ao método da Superclasse.
    public String toString() {
        return ("Tamanho do Lado = " + String.valueOf(lado) +
super.toString());
    }
}
```

B.3 Classe Circulo.Java

```
// Circulo.Java → Define as funcionalidades básicas do circulo

public class Circulo extends Figura {

    protected double raio;

    //Os quatro construtores

    public Circulo(double x, double y, double raio) {
        super(x, y);
        this.raio = raio;
    }

    public Circulo(double x, double y) {
        this(x, y, 1.0);
    }

    public Circulo(double raio) {
        this(0.0, 0.0, raio);
    }

    public Circulo() {
        this(0.0, 0.0, 1.0);
    }

    //Retorna o perimetro do circulo
    public double perimetro() {
        return 2 * Math.PI * raio;
    }

    //Retorna a area do circulo
    public double area() {
        return Math.PI * raio*raio;
    }

    // Adiciona informações do circulo ao método da Superclasse.
    public String toString() {
        return ("Raio do Circulo = " + String.valueOf(raio) +
super.toString());
    }
}
```

B.4 Aspecto Trace.Java

```
import Java.io.PrintStream;

//Trace.Java → Imprime as mensagens do tracing.
//As mensagens são impressas antes e depois dos pointcuts

abstract aspect Trace {

    /* -----
       ATRIBUTOS
       -----*/

    protected static PrintStream stream = System.err;
    protected static int profundidade = 0;

    /* -----
       MÉTODOS
       -----*/

    //Inicializa o stream
    public static void initStream(PrintStream s) {
        stream = s;
    }

    //imprime uma mensagem de entrada, deve ser chamado na entrada do
    //bloco a ser rastreado
    public static void traceEntrada(String str) {
        profundidade++;
        imprimeEntrada(str);
    }

    //imprime uma mensagem de saida, deve ser chamado na saida do
    //bloco a ser rastreado
    public static void traceSaida(String str) {
        imprimeSaida(str);
        profundidade--;
    }

    //string a ser impressa na entrada
    private static void imprimeEntrada(String str) {
        imprimeIndentacao();
        stream.println("--> " + str);
    }

    //string a ser impressa na saida
    private static void imprimeSaida(String str) {
        imprimeIndentacao();
        stream.println("<-- " + str);
    }

    //Controla a indentacao das mensagens de saida
    private static void imprimeIndentacao() {
        for (int i = 0; i < profundidade; i++)
            stream.print(" ");
    }
}
```

```

/* -----
   POINTCUTS!!!
   -----*/

//Classes → Será especificado em subclasse
abstract pointcut Classe();

//Os construtores das classes
pointcut Construtor(): Classe() && execution(new(..));

//As chamadas de método das classe+s
pointcut Metodo(): Classe() && execution(* *(..));

//Impressao das mensagens antes e depois dos pointcuts!
before(): Construtor() {
    traceEntrada("" + thisJoinPointStaticPart.getSignature());
}
after(): Construtor() {
    traceSaida("" + thisJoinPointStaticPart.getSignature());
}

before(): Metodo() {
    traceEntrada("" + thisJoinPointStaticPart.getSignature());
}
after(): Metodo() {
    traceSaida("" + thisJoinPointStaticPart.getSignature());
}
}

```

B.5 Aspecto TraceClasses.Java

```

/* TraceClasse.Java → subclasse da Trace.Java. Concretiza o abstract
pointcut, definindo as classes afetadas pelo Trace */

public aspect TraceClasses extends Trace {

    //definicao do pointcut Classe
    pointcut Classe(): within(Figura) ||
                      within(Circulo) ||
                      within(Quadrado);

    //Método main para testar o tracing
    public static void main(String[] args) {
        Trace.initStream(System.out);
        Exemplo.main(args);
    }
}

```

B.6 Classe Exemplo.Java

```
//Classe Exemplo com método main para testar o tracing

public class Exemplo {

    public static void main(String[] args) {

        Trace.initStream(System.out);

        Circulo c1 = new Circulo(2.0, 3.0, 2.0);

        Circulo c2 = new Circulo(4.0);

        Quadrado q1 = new Quadrado(1.0, 2.0);

        System.out.println("---\nc1.perimetro() = " + c1.perimetro());
        System.out.println("---\nc1.area() = " + c1.area());

        System.out.println("---\nq1.perimetro() = " + q1.perimetro());
        System.out.println("---\nq1.area() = " + q1.area());

        System.out.println("---\nc2.distancia(c1) = " + c2.distancia(c1));
        System.out.println("---\nq1.distancia(c1) = " + q1.distancia(c1));

        System.out.println("---\nc1.toString(): " + c1.toString());
        System.out.println("---\nc2.toString(): " + c2.toString());
        System.out.println("---\nq1.toString(): " + q1.toString());
    }
}
```

Apêndice C

Código Fonte Hello World – *AspectJ*

C.1 Classe HelloWorld.Java

```
public class HelloWorld {
    //Método main, que deve ser adequado afim de se fazer os testes
    public static void main(String[] args){
        dizPessoa("Olá Querido, tenha um Bom Dia!", "Igor");
    }
    //Método que imprime a mensagem + o nome da pessoa na tela
    public static void dizPessoa(String mensagem, String nome) {
        System.out.println(mensagem + ", " + nome);
    }
    //Método que imprime uma mensagem
    public static void diz(String message){
        System.out.println(message);
    }
}
```

C.2 Aspecto Saudacao.Java

```
public aspect Saudacao {
    //Pointcut que "corta" o código nos métodos que iniciam com diz
    //na classe HelloWorld
    pointcut Saudar() : call(* HelloWorld.diz*(..));
    //Mensagens a serem impressas antes e depois do ponto de combinacao
    before() : Saudar() {
        System.out.println("Olá!");
    }
    after() : Saudar() {
        System.out.println("Muito Obrigado!");
    }
}
```

C.3 Aspecto SaudJaponesa.Java

```
public aspect SaudJaponesa {
    //Pointcut no método dizPessoa da classe HelloWorld
    pointcut chamaPessoa (String pessoa):
        call(* HelloWorld.dizPessoa(String, String)) &&
        args(*, pessoa);
    //Inserção de código 'durante' a execução do ponto de combinacao
    void around(String pessoa) : chamaPessoa(pessoa) {
        proceed(pessoa + "-san");
    }
}
```